

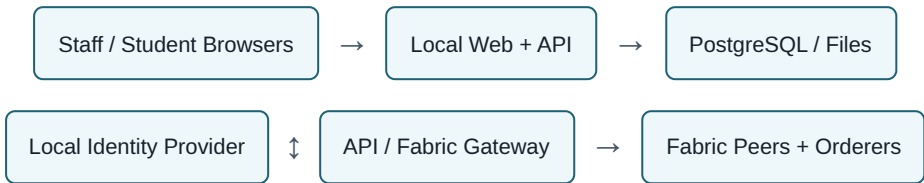
Blockchain for a School Education System

A 220-page full implementation handbook for a permissioned blockchain running entirely on a school-controlled local network, with no production dependency on the public Internet.

Reference architecture: Hyperledger Fabric + PostgreSQL + local identity + local file storage + local monitoring
Primary use cases: enrollment evidence, attendance audit, grade publication, transcripts, certificates, approvals, asset custody
Design style: privacy-first hybrid architecture; PII off-chain, tamper-evident hashes and approvals on-chain
Audience: school IT teams, blockchain developers, system architects, auditors, administrators, implementation vendors
Edition: August 2026

Executive Summary

The recommended school architecture is not a public cryptocurrency network. It is a permissioned, identity-controlled ledger on the school's LAN. The ledger is used only where tamper evidence, shared approval, and historical provenance provide value; ordinary school data remains in conventional systems.



Recommended operating model

On-chain

- Pseudonymous entity IDs
- Hashes of approved records
- Version and correction history
- Approval/endorsement evidence
- Certificate issue/revocation status
- Compact audit events

Off-chain

- Student names and contact data
- Detailed grades and attendance rows
- Health/disciplinary records
- Documents and photos
- Reports and analytics indexes
- Drafts and temporary data

Important: A blockchain does not automatically improve a school system. If a single trusted administrator controls all records and no independent approvals are needed, a well-secured database with append-only audit logs may be simpler. Use the blockchain layer only for records where the governance benefit is real.

Should a School Use Blockchain?

Use this page as the architecture decision record. A private ledger is justified when the school needs durable proof that important records passed through defined authorities and were not silently altered later.

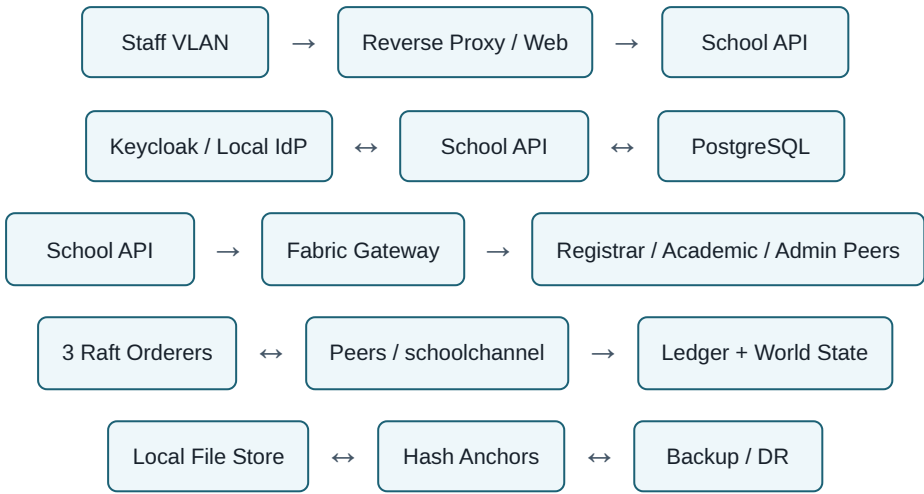
Question	Use blockchain when...	Prefer normal database when...
Who controls the record?	Several independent roles must approve or verify.	One trusted system owner is sufficient.
Correction model	History of corrections must remain provable.	Normal edit history/audit log is enough.
Data sensitivity	Only minimal pseudonymous evidence is written.	The full record must be erasable or frequently changed.
Availability	Distributed custody across multiple school authorities adds resilience.	Database HA is simpler and adequate.
External proof	Certificates/transcripts need cryptographic integrity proof.	No independent proof is required.

Recommended conclusion

For most schools, the best design is **hybrid**: keep the Student Information System and operational data in PostgreSQL; use Fabric as a controlled audit/approval/credential layer. This follows Fabric's permissioned identity and policy model rather than a public anonymous consensus model. [R1][R6]

Do not: create a token, mining process, public wallet, gas payment system, or public blockchain dependency for this use case. Those mechanisms solve different problems and add unnecessary operational and privacy risk.

Reference Architecture - Fully Local



Network services that must also be local

Core	Platform	Business
- DNS - NTP - DHCP - PKI/CA	- Container registry - Package repository - Monitoring/logging - Git/artifact mirror	- Identity - Database - File storage - Reporting

The application layer connects to Fabric through Gateway, which is the current simplified client path for transaction submission. [R4]

Ten Non-Negotiable Design Principles

1. **Privacy first:** keep PII and sensitive education records off-chain by default.
2. **Permissioned identities:** every privileged actor is known and revocable.
3. **No implicit LAN trust:** authenticate and authorize even inside the school network. [R13]
4. **Hybrid storage:** blockchain for evidence; database/files for operations.
5. **Multi-role approval:** map business authority to endorsement policy.
6. **Version, do not erase history:** corrections create new versions and reasons.
7. **No Internet dependency:** local DNS, NTP, identity, packages, registry, monitoring, and documentation.
8. **Recovery is designed, not improvised:** backup keys, ledger, database, identity, configuration, and files.
9. **Reproducible releases:** pinned versions, hashes, SBOMs, test evidence, rollback bundle.
10. **Usability matters:** teachers should not need blockchain knowledge to mark attendance or publish grades.

Architecture slogan: "The database runs the school; the blockchain proves the important decisions."

Security framework

Use a risk-management structure such as NIST CSF 2.0 to organize governance, identification, protection, detection, response, and recovery activities. [R16]

Privacy-First Data Placement Strategy

Data	Recommended location	Blockchain evidence
Student name, DOB, address, guardian	PostgreSQL - Restricted	Pseudonymous StudentUID + profile version hash
Attendance detail	PostgreSQL	Session/batch hash + approval/revision events
Grade details	PostgreSQL - Restricted	Approved grade-sheet hash + publication/correction events
Transcript PDF	Encrypted file store	Document hash + issuance metadata
Certificate	PDF/credential package	Credential ID + hash + issuer + status/revocation
Medical/disciplinary narrative	Special restricted system	Usually none
Private keys/passwords	Keystore/HSM/secret storage	Never

If the school is subject to FERPA, education-record access must be constrained to legitimate educational interests and identity/authentication must be reliable; local-only deployment does not remove privacy obligations. [R12]

Hashing is not anonymization. A hash of a predictable low-entropy value (for example a short student ID or known date) can sometimes be guessed. Hash canonical structured records with pseudonymous identifiers and, where appropriate, a secret salt/HMAC strategy for evidence that does not need public verification.

Recommended Technology Stack

Layer	Recommended choice	Reason
Permissioned ledger	Hyperledger Fabric v3.1.x pinned release	Known identities, policies, endorsement, private membership; v3.1.5 was current when this handbook was prepared. [R2]
Consensus	3-node Raft ordering	Simple CFT topology for a single-school trust domain. [R3]
Operational database	PostgreSQL 18.x pinned minor	Relational integrity, reporting, mature local operation. [R17]
Identity	Keycloak local server + Fabric CA/MSP	Web SSO/MFA separated from blockchain identity. [R18][R5]
Backend	Node.js/TypeScript or Go	Good Fabric Gateway support and local deployment.
Frontend	React/Vue/Svelte or server-rendered web	All assets can be bundled locally.
Container platform	Docker/Podman + Compose for small deployment	Simple repeatable local operations.
Monitoring	Local metrics/log stack	No cloud telemetry dependency.

Version rule: pin tested minor versions and container image digests. Do not automatically upgrade production from an Internet repository.

Table of Contents

1. Business Case, Scope, and Success Criteria	11-19
2. Stakeholders, Governance, and Operating Model	20-28
3. Local Network and Segmentation Architecture	29-37
4. Hardware, OS, Storage, and Capacity Planning	38-46
5. Hyperledger Fabric Architecture for a School	47-55
6. Fabric Network Build and Offline Installation	56-64
7. Consensus, Endorsement, Channels, and Chaincode Lifecycle	65-73
8. Identity, PKI, Authentication, and Authorization	74-82
9. Student Privacy, Data Classification, and Retention	83-91
10. Core Data Model, IDs, Hashing, and Ledger Patterns	92-100

Navigation: chapter titles are clickable in PDF viewers that preserve internal HTML links.

Table of Contents

11. Admissions and Student Master Workflow	101-109
12. Attendance and Timetable Ledger Workflow	110-118
13. Examinations, Grades, GPA, and Transcript Workflow	119-127
14. Certificates, Diplomas, Badges, and Offline Verification	128-136
15. Library, Fees, Assets, and Optional School Modules	137-145
16. Backend Services, APIs, Event Indexing, and Integration	146-154
17. Frontend, Local UX, Reporting, and Accessibility	155-163
18. Security Engineering and Threat Model	164-172
19. Testing, Performance, Reliability, and Quality Assurance	173-181
20. Operations, Monitoring, Backup, Disaster Recovery, and Upgrades	182-190
21. Source Code and Configuration Appendix	191-210
22. Rollout, SOPs, Glossary, and References	211-220

Navigation: chapter titles are clickable in PDF viewers that preserve internal HTML links.

How to Use This Handbook

Treat this document as an implementation playbook. Build the system in layers and require evidence at each gate.

Phase	Read	Exit evidence
1 - Decide	Pages 2-28	Approved scope, governance, data-classification and acceptance criteria
2 - Build LAN	Pages 29-46	Segmented network, local DNS/NTP, hardened hosts, backup infrastructure
3 - Build Fabric	Pages 47-82	3 orderers, org peers, channel, CAs/MSPs, Gateway, lifecycle process
4 - Protect data	Pages 83-109	Identity, RBAC, PII placement, encryption, retention, audit rules
5 - Implement modules	Pages 110-154	Student, attendance, grade, credential and optional module workflows
6 - Build app	Pages 155-172	API, event indexer, frontend, reports and reconciliation
7 - Secure/test/operate	Pages 173-190	Threat model, test pack, recovery evidence, monitoring/runbooks
8 - Implement code	Pages 191-210	Repository, config and code skeleton customized for the school
9 - Roll out	Pages 211-220	Pilot sign-off, SOPs, training, go-live and reference pack

Legal note: this is a technical architecture reference, not jurisdiction-specific legal advice. Map privacy, retention, student rights, accessibility, records-disclosure, and signature rules to your country/state before production.

Implementation Detail

1.1 School problems worth solving with a ledger

Identify high-value records where tamper evidence, multi-party approval, and a permanent audit trail are more important than raw transaction speed. Examples include enrollment approvals, grade publication, transcript issuance, certificate revocation, and staff authorization changes.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Start from a school-control problem, not from a desire to use blockchain.
- Assign one accountable business owner and one technical owner.
- Keep scope small enough that staff can learn and operate it.
- Define correction and appeal workflows before coding.

Implementation blueprint

- Write user stories with actor, precondition, action, approval, ledger evidence, off-chain record, and acceptance result.
- Create a RACI matrix and a transaction authority matrix.
- Give every high-risk workflow an exception path and rollback/compensation procedure.
- Record assumptions and unresolved policy questions in a decision log.

Security and privacy controls

- No shared privileged accounts.
- No direct database edits to bypass approval workflows.
- No production change without test evidence and approver identity.
- No PII on-chain unless specifically justified.

Acceptance / verification

- ✓ Demonstrate the normal workflow.
- ✓ Demonstrate unauthorized denial.
- ✓ Demonstrate correction/version history.
- ✓ Demonstrate audit reconstruction from a transaction ID.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

1.2 When blockchain is the wrong tool

Use a conventional database for mutable operational data, search-heavy workflows, medical/disciplinary notes, temporary drafts, and anything where one trusted administrator can safely be the sole authority. The recommended design is hybrid rather than blockchain-everything.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Start from a school-control problem, not from a desire to use blockchain.
- Assign one accountable business owner and one technical owner.
- Keep scope small enough that staff can learn and operate it.
- Define correction and appeal workflows before coding.

Implementation blueprint

- Write user stories with actor, precondition, action, approval, ledger evidence, off-chain record, and acceptance result.
- Create a RACI matrix and a transaction authority matrix.
- Give every high-risk workflow an exception path and rollback/compensation procedure.
- Record assumptions and unresolved policy questions in a decision log.

Security and privacy controls

- No shared privileged accounts.
- No direct database edits to bypass approval workflows.
- No production change without test evidence and approver identity.
- No PII on-chain unless specifically justified.

Acceptance / verification

- ✓ Demonstrate the normal workflow.
- ✓ Demonstrate unauthorized denial.
- ✓ Demonstrate correction/version history.
- ✓ Demonstrate audit reconstruction from a transaction ID.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

1.3 Define the local-only boundary

Document exactly what 'no Internet' means: no cloud APIs, no public DNS/NTP, no CDN assets, no SaaS authentication, no public blockchain RPC, and no automatic online updates. All production dependencies must resolve to school-controlled LAN services.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Start from a school-control problem, not from a desire to use blockchain.
- Assign one accountable business owner and one technical owner.
- Keep scope small enough that staff can learn and operate it.
- Define correction and appeal workflows before coding.

Implementation blueprint

- Write user stories with actor, precondition, action, approval, ledger evidence, off-chain record, and acceptance result.
- Create a RACI matrix and a transaction authority matrix.
- Give every high-risk workflow an exception path and rollback/compensation procedure.
- Record assumptions and unresolved policy questions in a decision log.

Security and privacy controls

- No shared privileged accounts.
- No direct database edits to bypass approval workflows.
- No production change without test evidence and approver identity.
- No PII on-chain unless specifically justified.

Acceptance / verification

- ✓ Demonstrate the normal workflow.
- ✓ Demonstrate unauthorized denial.
- ✓ Demonstrate correction/version history.
- ✓ Demonstrate audit reconstruction from a transaction ID.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

1.4 Functional scope for the first release

Limit release 1 to enrollment identity, attendance proof, grade publication approvals, transcript/certificate issuance, audit history, and local verification. Defer cafeteria, transportation, payroll, and advanced analytics until the core controls are stable.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Start from a school-control problem, not from a desire to use blockchain.
- Assign one accountable business owner and one technical owner.
- Keep scope small enough that staff can learn and operate it.
- Define correction and appeal workflows before coding.

Implementation blueprint

- Write user stories with actor, precondition, action, approval, ledger evidence, off-chain record, and acceptance result.
- Create a RACI matrix and a transaction authority matrix.
- Give every high-risk workflow an exception path and rollback/compensation procedure.
- Record assumptions and unresolved policy questions in a decision log.

Security and privacy controls

- No shared privileged accounts.
- No direct database edits to bypass approval workflows.
- No production change without test evidence and approver identity.
- No PII on-chain unless specifically justified.

Acceptance / verification

- ✓ Demonstrate the normal workflow.
- ✓ Demonstrate unauthorized denial.
- ✓ Demonstrate correction/version history.
- ✓ Demonstrate audit reconstruction from a transaction ID.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

1.5 Non-functional requirements

Set measurable targets for availability, response time, recovery point, recovery time, ledger commit latency, user concurrency, retention, auditability, and offline maintenance windows.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Start from a school-control problem, not from a desire to use blockchain.
- Assign one accountable business owner and one technical owner.
- Keep scope small enough that staff can learn and operate it.
- Define correction and appeal workflows before coding.

Implementation blueprint

- Write user stories with actor, precondition, action, approval, ledger evidence, off-chain record, and acceptance result.
- Create a RACI matrix and a transaction authority matrix.
- Give every high-risk workflow an exception path and rollback/compensation procedure.
- Record assumptions and unresolved policy questions in a decision log.

Security and privacy controls

- No shared privileged accounts.
- No direct database edits to bypass approval workflows.
- No production change without test evidence and approver identity.
- No PII on-chain unless specifically justified.

Acceptance / verification

- ✓ Demonstrate the normal workflow.
- ✓ Demonstrate unauthorized denial.
- ✓ Demonstrate correction/version history.
- ✓ Demonstrate audit reconstruction from a transaction ID.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

1.6 Success metrics

Measure duplicate-record reduction, unauthorized-change detection, certificate verification time, recovery-test success, teacher task completion time, and help-desk incidents rather than using blockchain transaction count as a success metric.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Start from a school-control problem, not from a desire to use blockchain.
- Assign one accountable business owner and one technical owner.
- Keep scope small enough that staff can learn and operate it.
- Define correction and appeal workflows before coding.

Implementation blueprint

- Write user stories with actor, precondition, action, approval, ledger evidence, off-chain record, and acceptance result.
- Create a RACI matrix and a transaction authority matrix.
- Give every high-risk workflow an exception path and rollback/compensation procedure.
- Record assumptions and unresolved policy questions in a decision log.

Security and privacy controls

- No shared privileged accounts.
- No direct database edits to bypass approval workflows.
- No production change without test evidence and approver identity.
- No PII on-chain unless specifically justified.

Acceptance / verification

- ✓ Demonstrate the normal workflow.
- ✓ Demonstrate unauthorized denial.
- ✓ Demonstrate correction/version history.
- ✓ Demonstrate audit reconstruction from a transaction ID.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

1.7 Assumptions and constraints register

Record class size, student count, staff count, device count, number of campuses, electrical reliability, IT staffing, school calendar, existing SIS/database, and whether district or examination-board nodes will participate.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Start from a school-control problem, not from a desire to use blockchain.
- Assign one accountable business owner and one technical owner.
- Keep scope small enough that staff can learn and operate it.
- Define correction and appeal workflows before coding.

Implementation blueprint

- Write user stories with actor, precondition, action, approval, ledger evidence, off-chain record, and acceptance result.
- Create a RACI matrix and a transaction authority matrix.
- Give every high-risk workflow an exception path and rollback/compensation procedure.
- Record assumptions and unresolved policy questions in a decision log.

Security and privacy controls

- No shared privileged accounts.
- No direct database edits to bypass approval workflows.
- No production change without test evidence and approver identity.
- No PII on-chain unless specifically justified.

Acceptance / verification

- ✓ Demonstrate the normal workflow.
- ✓ Demonstrate unauthorized denial.
- ✓ Demonstrate correction/version history.
- ✓ Demonstrate audit reconstruction from a transaction ID.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

1.8 MVP versus production boundary

Treat the first Docker Compose network as a controlled pilot. Production requires hardened hosts, dedicated certificates, backups, monitoring, change control, documented recovery, role separation, and security testing.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Start from a school-control problem, not from a desire to use blockchain.
- Assign one accountable business owner and one technical owner.
- Keep scope small enough that staff can learn and operate it.
- Define correction and appeal workflows before coding.

Implementation blueprint

- Write user stories with actor, precondition, action, approval, ledger evidence, off-chain record, and acceptance result.
- Create a RACI matrix and a transaction authority matrix.
- Give every high-risk workflow an exception path and rollback/compensation procedure.
- Record assumptions and unresolved policy questions in a decision log.

Security and privacy controls

- No shared privileged accounts.
- No direct database edits to bypass approval workflows.
- No production change without test evidence and approver identity.
- No PII on-chain unless specifically justified.

Acceptance / verification

- ✓ Demonstrate the normal workflow.
- ✓ Demonstrate unauthorized denial.
- ✓ Demonstrate correction/version history.
- ✓ Demonstrate audit reconstruction from a transaction ID.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

1.9 Acceptance definition for go-live

Go-live requires functional acceptance, security acceptance, restore testing, failover testing, audit review, training completion, signed governance rules, and a rollback plan. No single demo transaction is sufficient evidence of readiness.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Start from a school-control problem, not from a desire to use blockchain.
- Assign one accountable business owner and one technical owner.
- Keep scope small enough that staff can learn and operate it.
- Define correction and appeal workflows before coding.

Implementation blueprint

- Write user stories with actor, precondition, action, approval, ledger evidence, off-chain record, and acceptance result.
- Create a RACI matrix and a transaction authority matrix.
- Give every high-risk workflow an exception path and rollback/compensation procedure.
- Record assumptions and unresolved policy questions in a decision log.

Security and privacy controls

- No shared privileged accounts.
- No direct database edits to bypass approval workflows.
- No production change without test evidence and approver identity.
- No PII on-chain unless specifically justified.

Acceptance / verification

- ✓ Demonstrate the normal workflow.
- ✓ Demonstrate unauthorized denial.
- ✓ Demonstrate correction/version history.
- ✓ Demonstrate audit reconstruction from a transaction ID.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

2.1 Stakeholder map

Define responsibilities for principal, registrar, academic head, teachers, examination office, IT administrator, privacy officer, students, parents, and optional district/external credential authority.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Start from a school-control problem, not from a desire to use blockchain.
- Assign one accountable business owner and one technical owner.
- Keep scope small enough that staff can learn and operate it.
- Define correction and appeal workflows before coding.

Implementation blueprint

- Write user stories with actor, precondition, action, approval, ledger evidence, off-chain record, and acceptance result.
- Create a RACI matrix and a transaction authority matrix.
- Give every high-risk workflow an exception path and rollback/compensation procedure.
- Record assumptions and unresolved policy questions in a decision log.

Security and privacy controls

- No shared privileged accounts.
- No direct database edits to bypass approval workflows.
- No production change without test evidence and approver identity.
- No PII on-chain unless specifically justified.

Acceptance / verification

- ✓ Demonstrate the normal workflow.
- ✓ Demonstrate unauthorized denial.
- ✓ Demonstrate correction/version history.
- ✓ Demonstrate audit reconstruction from a transaction ID.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

2.2 Three-organization governance model

A practical single-school network can separate RegistrarOrg, AcademicOrg, and AdministrationOrg with independent MSP identities and peers. The separation should reflect real authority boundaries, not merely duplicate containers controlled by the same account.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Start from a school-control problem, not from a desire to use blockchain.
- Assign one accountable business owner and one technical owner.
- Keep scope small enough that staff can learn and operate it.
- Define correction and appeal workflows before coding.

Implementation blueprint

- Write user stories with actor, precondition, action, approval, ledger evidence, off-chain record, and acceptance result.
- Create a RACI matrix and a transaction authority matrix.
- Give every high-risk workflow an exception path and rollback/compensation procedure.
- Record assumptions and unresolved policy questions in a decision log.

Security and privacy controls

- No shared privileged accounts.
- No direct database edits to bypass approval workflows.
- No production change without test evidence and approver identity.
- No PII on-chain unless specifically justified.

Acceptance / verification

- ✓ Demonstrate the normal workflow.
- ✓ Demonstrate unauthorized denial.
- ✓ Demonstrate correction/version history.
- ✓ Demonstrate audit reconstruction from a transaction ID.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

2.3 Decision rights

Specify who can approve chaincode changes, channel configuration, certificate issuance, key revocation, emergency access, data retention changes, and production upgrades.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Start from a school-control problem, not from a desire to use blockchain.
- Assign one accountable business owner and one technical owner.
- Keep scope small enough that staff can learn and operate it.
- Define correction and appeal workflows before coding.

Implementation blueprint

- Write user stories with actor, precondition, action, approval, ledger evidence, off-chain record, and acceptance result.
- Create a RACI matrix and a transaction authority matrix.
- Give every high-risk workflow an exception path and rollback/compensation procedure.
- Record assumptions and unresolved policy questions in a decision log.

Security and privacy controls

- No shared privileged accounts.
- No direct database edits to bypass approval workflows.
- No production change without test evidence and approver identity.
- No PII on-chain unless specifically justified.

Acceptance / verification

- ✓ Demonstrate the normal workflow.
- ✓ Demonstrate unauthorized denial.
- ✓ Demonstrate correction/version history.
- ✓ Demonstrate audit reconstruction from a transaction ID.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

2.4 Endorsement governance

Map business approvals to endorsement policies. For example, published grades may require AcademicOrg plus RegistrarOrg; certificate issuance may require RegistrarOrg plus AdministrationOrg.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Start from a school-control problem, not from a desire to use blockchain.
- Assign one accountable business owner and one technical owner.
- Keep scope small enough that staff can learn and operate it.
- Define correction and appeal workflows before coding.

Implementation blueprint

- Write user stories with actor, precondition, action, approval, ledger evidence, off-chain record, and acceptance result.
- Create a RACI matrix and a transaction authority matrix.
- Give every high-risk workflow an exception path and rollback/compensation procedure.
- Record assumptions and unresolved policy questions in a decision log.

Security and privacy controls

- No shared privileged accounts.
- No direct database edits to bypass approval workflows.
- No production change without test evidence and approver identity.
- No PII on-chain unless specifically justified.

Acceptance / verification

- ✓ Demonstrate the normal workflow.
- ✓ Demonstrate unauthorized denial.
- ✓ Demonstrate correction/version history.
- ✓ Demonstrate audit reconstruction from a transaction ID.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

2.5 Change control board

Use a small change board for production ledger rules. Every chaincode/config change should have a ticket, risk assessment, test evidence, rollback procedure, approvals, and release hash.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Start from a school-control problem, not from a desire to use blockchain.
- Assign one accountable business owner and one technical owner.
- Keep scope small enough that staff can learn and operate it.
- Define correction and appeal workflows before coding.

Implementation blueprint

- Write user stories with actor, precondition, action, approval, ledger evidence, off-chain record, and acceptance result.
- Create a RACI matrix and a transaction authority matrix.
- Give every high-risk workflow an exception path and rollback/compensation procedure.
- Record assumptions and unresolved policy questions in a decision log.

Security and privacy controls

- No shared privileged accounts.
- No direct database edits to bypass approval workflows.
- No production change without test evidence and approver identity.
- No PII on-chain unless specifically justified.

Acceptance / verification

- ✓ Demonstrate the normal workflow.
- ✓ Demonstrate unauthorized denial.
- ✓ Demonstrate correction/version history.
- ✓ Demonstrate audit reconstruction from a transaction ID.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

2.6 Segregation of duties

Prevent one person from creating a student, assigning a final grade, issuing a certificate, changing access policy, and deleting audit evidence. Split high-impact actions among independent roles.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Start from a school-control problem, not from a desire to use blockchain.
- Assign one accountable business owner and one technical owner.
- Keep scope small enough that staff can learn and operate it.
- Define correction and appeal workflows before coding.

Implementation blueprint

- Write user stories with actor, precondition, action, approval, ledger evidence, off-chain record, and acceptance result.
- Create a RACI matrix and a transaction authority matrix.
- Give every high-risk workflow an exception path and rollback/compensation procedure.
- Record assumptions and unresolved policy questions in a decision log.

Security and privacy controls

- No shared privileged accounts.
- No direct database edits to bypass approval workflows.
- No production change without test evidence and approver identity.
- No PII on-chain unless specifically justified.

Acceptance / verification

- ✓ Demonstrate the normal workflow.
- ✓ Demonstrate unauthorized denial.
- ✓ Demonstrate correction/version history.
- ✓ Demonstrate audit reconstruction from a transaction ID.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

2.7 Audit committee workflow

Define monthly review of identity changes, privileged transactions, certificate revocations, failed login trends, backup health, and policy exceptions.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Start from a school-control problem, not from a desire to use blockchain.
- Assign one accountable business owner and one technical owner.
- Keep scope small enough that staff can learn and operate it.
- Define correction and appeal workflows before coding.

Implementation blueprint

- Write user stories with actor, precondition, action, approval, ledger evidence, off-chain record, and acceptance result.
- Create a RACI matrix and a transaction authority matrix.
- Give every high-risk workflow an exception path and rollback/compensation procedure.
- Record assumptions and unresolved policy questions in a decision log.

Security and privacy controls

- No shared privileged accounts.
- No direct database edits to bypass approval workflows.
- No production change without test evidence and approver identity.
- No PII on-chain unless specifically justified.

Acceptance / verification

- ✓ Demonstrate the normal workflow.
- ✓ Demonstrate unauthorized denial.
- ✓ Demonstrate correction/version history.
- ✓ Demonstrate audit reconstruction from a transaction ID.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

2.8 Policy documents

Create policies for acceptable use, identity lifecycle, grade correction, certificate revocation, privacy, backup, incident response, software updates, and media handling.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Start from a school-control problem, not from a desire to use blockchain.
- Assign one accountable business owner and one technical owner.
- Keep scope small enough that staff can learn and operate it.
- Define correction and appeal workflows before coding.

Implementation blueprint

- Write user stories with actor, precondition, action, approval, ledger evidence, off-chain record, and acceptance result.
- Create a RACI matrix and a transaction authority matrix.
- Give every high-risk workflow an exception path and rollback/compensation procedure.
- Record assumptions and unresolved policy questions in a decision log.

Security and privacy controls

- No shared privileged accounts.
- No direct database edits to bypass approval workflows.
- No production change without test evidence and approver identity.
- No PII on-chain unless specifically justified.

Acceptance / verification

- ✓ Demonstrate the normal workflow.
- ✓ Demonstrate unauthorized denial.
- ✓ Demonstrate correction/version history.
- ✓ Demonstrate audit reconstruction from a transaction ID.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

2.9 Governance failure scenarios

Plan for staff departure, compromised administrator, disputed grade, invalid certificate, corrupted node, unavailable registrar, and emergency school closure. Governance rules must still work when one role or server is unavailable.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Start from a school-control problem, not from a desire to use blockchain.
- Assign one accountable business owner and one technical owner.
- Keep scope small enough that staff can learn and operate it.
- Define correction and appeal workflows before coding.

Implementation blueprint

- Write user stories with actor, precondition, action, approval, ledger evidence, off-chain record, and acceptance result.
- Create a RACI matrix and a transaction authority matrix.
- Give every high-risk workflow an exception path and rollback/compensation procedure.
- Record assumptions and unresolved policy questions in a decision log.

Security and privacy controls

- No shared privileged accounts.
- No direct database edits to bypass approval workflows.
- No production change without test evidence and approver identity.
- No PII on-chain unless specifically justified.

Acceptance / verification

- ✓ Demonstrate the normal workflow.
- ✓ Demonstrate unauthorized denial.
- ✓ Demonstrate correction/version history.
- ✓ Demonstrate audit reconstruction from a transaction ID.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

3.1 Reference LAN topology

Use separate management, blockchain, application, database/storage, staff, student, and backup VLANs. Route between them only through explicit firewall rules; do not treat a physically local network as inherently trusted.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Offline does not mean trusted.
- Segment users from servers and servers from backups.
- Use explicit allowlists and local infrastructure services.
- Design for switch, DNS, and time-service failures.

Implementation blueprint

- Create VLAN/IP/DNS/port diagrams and keep them under change control.
- Use redundant core networking for production if budget permits.
- Document every listening service and owner.
- Keep management interfaces reachable only from the management VLAN.

Security and privacy controls

- Default-deny routing.
- TLS even inside the LAN.
- No public DNS/NTP dependencies.
- No student-to-server management paths.

Acceptance / verification

- ✓ Port scan from each VLAN.
- ✓ Stop one DNS/time source.
- ✓ Verify no Internet egress.
- ✓ Validate firewall logs for denied lateral movement.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

3.2 Example IP plan

Reserve stable RFC1918 subnets such as 10.10.10.0/24 management, 10.10.20.0/24 blockchain, 10.10.30.0/24 application, 10.10.40.0/24 data, 10.10.50.0/24 staff, 10.10.60.0/24 student, and 10.10.70.0/24 backup.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Offline does not mean trusted.
- Segment users from servers and servers from backups.
- Use explicit allowlists and local infrastructure services.
- Design for switch, DNS, and time-service failures.

Implementation blueprint

- Create VLAN/IP/DNS/port diagrams and keep them under change control.
- Use redundant core networking for production if budget permits.
- Document every listening service and owner.
- Keep management interfaces reachable only from the management VLAN.

Security and privacy controls

- Default-deny routing.
- TLS even inside the LAN.
- No public DNS/NTP dependencies.
- No student-to-server management paths.

Acceptance / verification

- ✓ Port scan from each VLAN.
- ✓ Stop one DNS/time source.
- ✓ Verify no Internet egress.
- ✓ Validate firewall logs for denied lateral movement.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

3.3 Firewall policy

Default-deny inter-VLAN traffic. Permit only specific source/destination/service combinations such as app-to-Gateway gRPC, app-to-PostgreSQL, peers-to-orderers, monitoring-to-exporters, and admins-to-management interfaces.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Offline does not mean trusted.
- Segment users from servers and servers from backups.
- Use explicit allowlists and local infrastructure services.
- Design for switch, DNS, and time-service failures.

Implementation blueprint

- Create VLAN/IP/DNS/port diagrams and keep them under change control.
- Use redundant core networking for production if budget permits.
- Document every listening service and owner.
- Keep management interfaces reachable only from the management VLAN.

Security and privacy controls

- Default-deny routing.
- TLS even inside the LAN.
- No public DNS/NTP dependencies.
- No student-to-server management paths.

Acceptance / verification

- ✓ Port scan from each VLAN.
- ✓ Stop one DNS/time source.
- ✓ Verify no Internet egress.
- ✓ Validate firewall logs for denied lateral movement.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

3.4 Local DNS

Operate redundant authoritative/caching DNS on the LAN with names such as app.school.lan, peer0.registrar.school.lan, orderer1.school.lan, id.school.lan, db.school.lan, and backup.school.lan.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Offline does not mean trusted.
- Segment users from servers and servers from backups.
- Use explicit allowlists and local infrastructure services.
- Design for switch, DNS, and time-service failures.

Implementation blueprint

- Create VLAN/IP/DNS/port diagrams and keep them under change control.
- Use redundant core networking for production if budget permits.
- Document every listening service and owner.
- Keep management interfaces reachable only from the management VLAN.

Security and privacy controls

- Default-deny routing.
- TLS even inside the LAN.
- No public DNS/NTP dependencies.
- No student-to-server management paths.

Acceptance / verification

- ✓ Port scan from each VLAN.
- ✓ Stop one DNS/time source.
- ✓ Verify no Internet egress.
- ✓ Validate firewall logs for denied lateral movement.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

3.5 Local time synchronization

Use an internal NTP hierarchy. One or two protected time masters serve servers; clients sync from local time servers. Correct timestamps are essential for certificates, logs, audit events, and incident investigation.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Offline does not mean trusted.
- Segment users from servers and servers from backups.
- Use explicit allowlists and local infrastructure services.
- Design for switch, DNS, and time-service failures.

Implementation blueprint

- Create VLAN/IP/DNS/port diagrams and keep them under change control.
- Use redundant core networking for production if budget permits.
- Document every listening service and owner.
- Keep management interfaces reachable only from the management VLAN.

Security and privacy controls

- Default-deny routing.
- TLS even inside the LAN.
- No public DNS/NTP dependencies.
- No student-to-server management paths.

Acceptance / verification

- ✓ Port scan from each VLAN.
- ✓ Stop one DNS/time source.
- ✓ Verify no Internet egress.
- ✓ Validate firewall logs for denied lateral movement.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

3.6 No default Internet route

Production server VLANs should have no default route to the Internet. If a maintenance gateway is temporarily enabled, use a documented window, allowlist, packet logging, and immediate route removal afterward.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Offline does not mean trusted.
- Segment users from servers and servers from backups.
- Use explicit allowlists and local infrastructure services.
- Design for switch, DNS, and time-service failures.

Implementation blueprint

- Create VLAN/IP/DNS/port diagrams and keep them under change control.
- Use redundant core networking for production if budget permits.
- Document every listening service and owner.
- Keep management interfaces reachable only from the management VLAN.

Security and privacy controls

- Default-deny routing.
- TLS even inside the LAN.
- No public DNS/NTP dependencies.
- No student-to-server management paths.

Acceptance / verification

- ✓ Port scan from each VLAN.
- ✓ Stop one DNS/time source.
- ✓ Verify no Internet egress.
- ✓ Validate firewall logs for denied lateral movement.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

3.7 Staff and student device isolation

Student devices must not reach Fabric peers, orderers, CAs, PostgreSQL, or backup systems. They should reach only approved web/API endpoints through the application tier.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Offline does not mean trusted.
- Segment users from servers and servers from backups.
- Use explicit allowlists and local infrastructure services.
- Design for switch, DNS, and time-service failures.

Implementation blueprint

- Create VLAN/IP/DNS/port diagrams and keep them under change control.
- Use redundant core networking for production if budget permits.
- Document every listening service and owner.
- Keep management interfaces reachable only from the management VLAN.

Security and privacy controls

- Default-deny routing.
- TLS even inside the LAN.
- No public DNS/NTP dependencies.
- No student-to-server management paths.

Acceptance / verification

- ✓ Port scan from each VLAN.
- ✓ Stop one DNS/time source.
- ✓ Verify no Internet egress.
- ✓ Validate firewall logs for denied lateral movement.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

3.8 Wireless design

Use separate staff, student, guest, and device SSIDs mapped to VLANs. The blockchain itself remains on wired server networks; Wi-Fi clients access only application services.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Offline does not mean trusted.
- Segment users from servers and servers from backups.
- Use explicit allowlists and local infrastructure services.
- Design for switch, DNS, and time-service failures.

Implementation blueprint

- Create VLAN/IP/DNS/port diagrams and keep them under change control.
- Use redundant core networking for production if budget permits.
- Document every listening service and owner.
- Keep management interfaces reachable only from the management VLAN.

Security and privacy controls

- Default-deny routing.
- TLS even inside the LAN.
- No public DNS/NTP dependencies.
- No student-to-server management paths.

Acceptance / verification

- ✓ Port scan from each VLAN.
- ✓ Stop one DNS/time source.
- ✓ Verify no Internet egress.
- ✓ Validate firewall logs for denied lateral movement.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

3.9 LAN failure and degraded-mode plan

Define what happens if a switch, firewall, DNS server, or time server fails. Critical services should have static or redundant dependencies and documented manual fallback procedures.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Offline does not mean trusted.
- Segment users from servers and servers from backups.
- Use explicit allowlists and local infrastructure services.
- Design for switch, DNS, and time-service failures.

Implementation blueprint

- Create VLAN/IP/DNS/port diagrams and keep them under change control.
- Use redundant core networking for production if budget permits.
- Document every listening service and owner.
- Keep management interfaces reachable only from the management VLAN.

Security and privacy controls

- Default-deny routing.
- TLS even inside the LAN.
- No public DNS/NTP dependencies.
- No student-to-server management paths.

Acceptance / verification

- ✓ Port scan from each VLAN.
- ✓ Stop one DNS/time source.
- ✓ Verify no Internet egress.
- ✓ Validate firewall logs for denied lateral movement.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

4.1 Minimum pilot hardware

For a pilot, use three server-class or business-class hosts with 4-8 CPU cores, 16-32 GB RAM, SSD storage, redundant power where possible, and UPS protection. Avoid placing the entire network on one laptop.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Availability requires independent failure domains.
- RAID is not backup.
- Power and cooling are part of the system design.
- Capacity planning must include ledger growth and school-year peaks.

Implementation blueprint

- Inventory CPU/RAM/disk/network/UPS serials and warranty dates.
- Reserve 30-40% free storage headroom.
- Separate production, backup, and admin credentials.
- Keep a tested spare disk and restore media.

Security and privacy controls

- Lock server rooms.
- Encrypt disks and removable backups.
- Disable unused physical/remote-management ports.
- Record chain of custody for retired drives.

Acceptance / verification

- ✓ Power-loss test.
- ✓ Disk-failure alert test.
- ✓ Cold restart test.
- ✓ Capacity alarm test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

4.2 Production physical-node layout

Place three orderers on three separate physical hosts. Run organization peers on at least two hosts when budget permits, so loss of one machine does not remove all endorsement capability.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Availability requires independent failure domains.
- RAID is not backup.
- Power and cooling are part of the system design.
- Capacity planning must include ledger growth and school-year peaks.

Implementation blueprint

- Inventory CPU/RAM/disk/network/UPS serials and warranty dates.
- Reserve 30-40% free storage headroom.
- Separate production, backup, and admin credentials.
- Keep a tested spare disk and restore media.

Security and privacy controls

- Lock server rooms.
- Encrypt disks and removable backups.
- Disable unused physical/remote-management ports.
- Record chain of custody for retired drives.

Acceptance / verification

- ✓ Power-loss test.
- ✓ Disk-failure alert test.
- ✓ Cold restart test.
- ✓ Capacity alarm test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

4.3 Storage tiers

Use fast SSD/NVMe for ledger/world-state and PostgreSQL, separate capacity storage for documents and backups, and encrypted removable media for offline backup copies.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Availability requires independent failure domains.
- RAID is not backup.
- Power and cooling are part of the system design.
- Capacity planning must include ledger growth and school-year peaks.

Implementation blueprint

- Inventory CPU/RAM/disk/network/UPS serials and warranty dates.
- Reserve 30-40% free storage headroom.
- Separate production, backup, and admin credentials.
- Keep a tested spare disk and restore media.

Security and privacy controls

- Lock server rooms.
- Encrypt disks and removable backups.
- Disable unused physical/remote-management ports.
- Record chain of custody for retired drives.

Acceptance / verification

- ✓ Power-loss test.
- ✓ Disk-failure alert test.
- ✓ Cold restart test.
- ✓ Capacity alarm test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

4.4 RAID and filesystem choices

RAID improves availability but is not a backup. Use a journaling filesystem, monitor SMART health, reserve free space, and avoid storing databases on consumer USB flash media.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Availability requires independent failure domains.
- RAID is not backup.
- Power and cooling are part of the system design.
- Capacity planning must include ledger growth and school-year peaks.

Implementation blueprint

- Inventory CPU/RAM/disk/network/UPS serials and warranty dates.
- Reserve 30-40% free storage headroom.
- Separate production, backup, and admin credentials.
- Keep a tested spare disk and restore media.

Security and privacy controls

- Lock server rooms.
- Encrypt disks and removable backups.
- Disable unused physical/remote-management ports.
- Record chain of custody for retired drives.

Acceptance / verification

- ✓ Power-loss test.
- ✓ Disk-failure alert test.
- ✓ Cold restart test.
- ✓ Capacity alarm test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

4.5 UPS and power events

Size UPS runtime to support clean shutdown or generator switchover. Integrate low-battery shutdown scripts and test them during scheduled maintenance.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Availability requires independent failure domains.
- RAID is not backup.
- Power and cooling are part of the system design.
- Capacity planning must include ledger growth and school-year peaks.

Implementation blueprint

- Inventory CPU/RAM/disk/network/UPS serials and warranty dates.
- Reserve 30-40% free storage headroom.
- Separate production, backup, and admin credentials.
- Keep a tested spare disk and restore media.

Security and privacy controls

- Lock server rooms.
- Encrypt disks and removable backups.
- Disable unused physical/remote-management ports.
- Record chain of custody for retired drives.

Acceptance / verification

- ✓ Power-loss test.
- ✓ Disk-failure alert test.
- ✓ Cold restart test.
- ✓ Capacity alarm test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

4.6 Server operating system baseline

Use a currently supported Linux LTS release, minimal package set, secure boot where practical, host firewall, automatic local log rotation, disabled unused services, and hardened SSH configuration.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Availability requires independent failure domains.
- RAID is not backup.
- Power and cooling are part of the system design.
- Capacity planning must include ledger growth and school-year peaks.

Implementation blueprint

- Inventory CPU/RAM/disk/network/UPS serials and warranty dates.
- Reserve 30-40% free storage headroom.
- Separate production, backup, and admin credentials.
- Keep a tested spare disk and restore media.

Security and privacy controls

- Lock server rooms.
- Encrypt disks and removable backups.
- Disable unused physical/remote-management ports.
- Record chain of custody for retired drives.

Acceptance / verification

- ✓ Power-loss test.
- ✓ Disk-failure alert test.
- ✓ Cold restart test.
- ✓ Capacity alarm test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

4.7 Container runtime strategy

Docker Compose is appropriate for a small on-premises deployment if operational discipline is strong. Pin image digests, keep a local registry, and never rely on the Internet to pull images at production start.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Availability requires independent failure domains.
- RAID is not backup.
- Power and cooling are part of the system design.
- Capacity planning must include ledger growth and school-year peaks.

Implementation blueprint

- Inventory CPU/RAM/disk/network/UPS serials and warranty dates.
- Reserve 30-40% free storage headroom.
- Separate production, backup, and admin credentials.
- Keep a tested spare disk and restore media.

Security and privacy controls

- Lock server rooms.
- Encrypt disks and removable backups.
- Disable unused physical/remote-management ports.
- Record chain of custody for retired drives.

Acceptance / verification

- ✓ Power-loss test.
- ✓ Disk-failure alert test.
- ✓ Cold restart test.
- ✓ Capacity alarm test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

4.8 Capacity model

Estimate annual transactions from students x school days x events. Attendance can dominate write volume; design it as compact events while keeping large documents off-chain.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Availability requires independent failure domains.
- RAID is not backup.
- Power and cooling are part of the system design.
- Capacity planning must include ledger growth and school-year peaks.

Implementation blueprint

- Inventory CPU/RAM/disk/network/UPS serials and warranty dates.
- Reserve 30-40% free storage headroom.
- Separate production, backup, and admin credentials.
- Keep a tested spare disk and restore media.

Security and privacy controls

- Lock server rooms.
- Encrypt disks and removable backups.
- Disable unused physical/remote-management ports.
- Record chain of custody for retired drives.

Acceptance / verification

- ✓ Power-loss test.
- ✓ Disk-failure alert test.
- ✓ Cold restart test.
- ✓ Capacity alarm test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

4.9 Bill of materials and spare capacity

Budget for spare disks, UPS batteries, a cold-spares host, encrypted backup drives, managed switches, firewall, dedicated admin workstation, and replacement certificates/tokens.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Availability requires independent failure domains.
- RAID is not backup.
- Power and cooling are part of the system design.
- Capacity planning must include ledger growth and school-year peaks.

Implementation blueprint

- Inventory CPU/RAM/disk/network/UPS serials and warranty dates.
- Reserve 30-40% free storage headroom.
- Separate production, backup, and admin credentials.
- Keep a tested spare disk and restore media.

Security and privacy controls

- Lock server rooms.
- Encrypt disks and removable backups.
- Disable unused physical/remote-management ports.
- Record chain of custody for retired drives.

Acceptance / verification

- ✓ Power-loss test.
- ✓ Disk-failure alert test.
- ✓ Cold restart test.
- ✓ Capacity alarm test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

5.1 Why permissioned Fabric

A school needs known identities, controlled participation, policy-based endorsement, private membership, and no mining. Hyperledger Fabric is designed as a permissioned distributed ledger rather than a public anonymous chain. [R1][R5]

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

5.2 Recommended Fabric version policy

Pin a tested v3.1.x release for the deployment bundle and document the exact image digests. Fabric v3.x adds BFT ordering as an option, while Raft remains the practical default for a trusted single-school LAN. [R2][R3]

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

5.3 Peers and world state

Peers maintain the blockchain ledger and world state, endorse chaincode execution, and expose Gateway services. Separate organization peers make policy boundaries explicit.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

5.4 Ordering service

Use three Raft orderers across separate hosts for a small production network. A majority must be available for progress; maintenance must preserve quorum. [R3]

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

5.5 Channels

Start with one schoolchannel unless there is a genuine membership/privacy boundary requiring another channel. Too many channels create operational overhead and duplicated configuration.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

5.6 MSP and organizational identity

Each organization has an MSP defining trusted certificate authorities, node identities, admins, and clients. X.509 certificates are the default Fabric identity mechanism. [R5]

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

5.7 Fabric Gateway

Applications connect to a peer Gateway rather than manually orchestrating endorsements. This simplifies client logic and narrows the required network paths. [R4]

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

5.8 Ledger versus database

The Fabric ledger stores ordered, tamper-evident transactions and current world state. PostgreSQL remains the operational query store for PII-heavy, mutable, reporting-oriented data. [R10]

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

5.9 School reference network

RegistrarOrg, AcademicOrg, AdministrationOrg, three orderers, one channel, Fabric CAs/intermediate CAs, application gateway services, PostgreSQL, file storage, Keycloak, monitoring, and backup are the baseline components.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

6.1 Connected staging workstation

Use a separate Internet-connected staging machine to download Fabric binaries/images, OS packages, container images, Node dependencies, and documentation. Verify signatures/checksums before transfer to the school environment.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

6.2 Offline software bundle

Create a versioned bundle containing Fabric binaries, Fabric CA, container images, chaincode dependencies, application packages, SBOMs, hashes, licenses, and installation instructions.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

6.3 Local container registry

Import signed/pinned images into a LAN registry. Production Compose files should reference the local registry by digest so startup never depends on public registries.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

6.4 Create root and intermediate CAs

Keep the root CA offline. Use intermediate enrollment/TLS CAs for day-to-day issuance. Back up CA databases, certificates, signing keys, and configuration separately.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

6.5 Generate organization MSPs

Enroll admins, peers, orderers, and application identities using Fabric CA. Use Node OUs/roles and verify certificate paths before channel creation.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

6.6 Build the ordering service

Create the three-node Raft ordering service, distribute TLS certificates, configure consenters, set resource limits, and test quorum by stopping one orderer. [R3]

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

6.7 Deploy peers

Deploy Registrar, Academic, and Administration peers with dedicated data volumes and TLS. Enable Gateway on the peers used by applications.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

6.8 Create schoolchannel

Define organization MSPs, policies, capabilities, anchor peers, orderer endpoints, and application policies. Generate the genesis/config and join orderers/peers using the supported administration flow.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

6.9 Production network validation

Confirm TLS, channel membership, block delivery, peer ledger height, Gateway transaction flow, organization endorsement, log collection, metrics, and restart persistence before deploying school chaincode.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

7.1 Raft quorum behavior

With three orderers, two must be healthy to continue ordering. Place nodes on independent hosts and power circuits where feasible. Test one-node failure and recovery before go-live.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

7.2 When BFT is justified

Consider Fabric BFT ordering only if administrative domains may behave maliciously rather than merely fail. BFT adds operational complexity and should not be chosen just because it sounds more secure. [R2]

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

7.3 Business endorsement matrix

Create a table mapping each ledger transaction to required organizations. Low-risk queries need no endorsement policy change; final grades and certificates should require multi-organization approval.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

7.4 State-based endorsement

Use state-based endorsement selectively for records needing stronger rules, such as a certificate record that requires registrar plus principal approval even if the chaincode default is less strict.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

7.5 Chaincode packaging

Build reproducible chaincode packages from locked dependencies. Record package hash, source commit, build environment, test evidence, and approvers.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

7.6 Approve and commit definitions

Each required organization approves the same chaincode definition, version, sequence, endorsement policy, and collection configuration before commit. [R8][R9]

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

7.7 Chaincode upgrade procedure

Increment sequence, run migration tests, validate backward-compatible state handling, approve the new definition, commit during a maintenance window, and retain rollback artifacts.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

7.8 Event strategy

Emit compact chaincode events for enrollment approval, attendance finalization, grade publication, certificate issue/revocation, and access-policy changes. Backend indexers consume events to update reporting stores.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

7.9 Channel configuration changes

Treat config updates as high-risk governed operations. Store before/after configs, signatures, ticket IDs, approvers, and recovery notes.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Use permissioned identities and policy-driven approval.
- Prefer simple topology before advanced consensus features.
- Keep chaincode deterministic.
- Treat lifecycle/config changes as governance events.

Implementation blueprint

- Pin exact Fabric/CA/container versions and hashes.
- Separate org MSP material and TLS material.
- Use Gateway for application transaction submission.
- Document channel, policies, orderers, peers, and chaincode sequence.

Security and privacy controls

- Protect CA and admin private keys.
- Require multi-org endorsement for high-risk records.
- Do not expose peer/orderer ports to student networks.
- Audit channel/config changes.

Acceptance / verification

- ✓ Peer restart persistence.
- ✓ Orderer quorum test.
- ✓ Endorsement deny test.
- ✓ Gateway submit/evaluate/event test.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

8.1 Human identity architecture

Use a local identity provider such as Keycloak for web login and MFA, while Fabric CA/MSP controls blockchain identities. Keep the two systems linked by stable internal user IDs rather than email addresses.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Human identity, device identity, and blockchain identity are related but distinct.
- Authenticate strongly and authorize narrowly.
- Use individual identities for high-impact actions.
- Revoke quickly when roles change.

Implementation blueprint

- Map Keycloak subject ID to school staff ID and optional Fabric enrollment ID.
- Define role/attribute claims centrally.
- Document certificate issuance, renewal, revocation, and lost-token processes.
- Use short session lifetimes for privileged users.

Security and privacy controls

- MFA for privileged roles.
- No shared teacher/admin accounts.
- Separate CA admin from application admin.
- Protect recovery codes and token seeds.

Acceptance / verification

- ✓ Revoked user cannot submit.
- ✓ Changed role takes effect.
- ✓ Expired certificate handled cleanly.
- ✓ Shared-device logout removes session data.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

8.2 Staff versus student identities

Teachers, registrar staff, principals, and blockchain administrators need high-assurance identities. Students generally use application accounts; only workflows requiring direct cryptographic non-repudiation need individual Fabric client certificates.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Human identity, device identity, and blockchain identity are related but distinct.
- Authenticate strongly and authorize narrowly.
- Use individual identities for high-impact actions.
- Revoke quickly when roles change.

Implementation blueprint

- Map Keycloak subject ID to school staff ID and optional Fabric enrollment ID.
- Define role/attribute claims centrally.
- Document certificate issuance, renewal, revocation, and lost-token processes.
- Use short session lifetimes for privileged users.

Security and privacy controls

- MFA for privileged roles.
- No shared teacher/admin accounts.
- Separate CA admin from application admin.
- Protect recovery codes and token seeds.

Acceptance / verification

- ✓ Revoked user cannot submit.
- ✓ Changed role takes effect.
- ✓ Expired certificate handled cleanly.
- ✓ Shared-device logout removes session data.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

8.3 Per-user Fabric identities

For high-assurance actions, issue personal Fabric client certificates to staff and sign transaction proposals under the staff identity. Avoid a single shared application certificate for all sensitive actions.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Human identity, device identity, and blockchain identity are related but distinct.
- Authenticate strongly and authorize narrowly.
- Use individual identities for high-impact actions.
- Revoke quickly when roles change.

Implementation blueprint

- Map Keycloak subject ID to school staff ID and optional Fabric enrollment ID.
- Define role/attribute claims centrally.
- Document certificate issuance, renewal, revocation, and lost-token processes.
- Use short session lifetimes for privileged users.

Security and privacy controls

- MFA for privileged roles.
- No shared teacher/admin accounts.
- Separate CA admin from application admin.
- Protect recovery codes and token seeds.

Acceptance / verification

- ✓ Revoked user cannot submit.
- ✓ Changed role takes effect.
- ✓ Expired certificate handled cleanly.
- ✓ Shared-device logout removes session data.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

8.4 Service identities

Backend services may use dedicated service certificates for automated events, but chaincode must distinguish automated services from humans and limit their functions.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Human identity, device identity, and blockchain identity are related but distinct.
- Authenticate strongly and authorize narrowly.
- Use individual identities for high-impact actions.
- Revoke quickly when roles change.

Implementation blueprint

- Map Keycloak subject ID to school staff ID and optional Fabric enrollment ID.
- Define role/attribute claims centrally.
- Document certificate issuance, renewal, revocation, and lost-token processes.
- Use short session lifetimes for privileged users.

Security and privacy controls

- MFA for privileged roles.
- No shared teacher/admin accounts.
- Separate CA admin from application admin.
- Protect recovery codes and token seeds.

Acceptance / verification

- ✓ Revoked user cannot submit.
- ✓ Changed role takes effect.
- ✓ Expired certificate handled cleanly.
- ✓ Shared-device logout removes session data.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

8.5 Role-based access control

Define roles such as registrar.write, teacher.attendance, teacher.grade-draft, examiner.publish, principal.approve, auditor.read, it.network-admin, and ca-admin. Deny by default.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Human identity, device identity, and blockchain identity are related but distinct.
- Authenticate strongly and authorize narrowly.
- Use individual identities for high-impact actions.
- Revoke quickly when roles change.

Implementation blueprint

- Map Keycloak subject ID to school staff ID and optional Fabric enrollment ID.
- Define role/attribute claims centrally.
- Document certificate issuance, renewal, revocation, and lost-token processes.
- Use short session lifetimes for privileged users.

Security and privacy controls

- MFA for privileged roles.
- No shared teacher/admin accounts.
- Separate CA admin from application admin.
- Protect recovery codes and token seeds.

Acceptance / verification

- ✓ Revoked user cannot submit.
- ✓ Changed role takes effect.
- ✓ Expired certificate handled cleanly.
- ✓ Shared-device logout removes session data.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

8.6 Attribute-based rules

Use certificate attributes or application claims for department, campus, subject, and job function when access depends on context. Do not encode volatile personal data in long-lived certificates.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Human identity, device identity, and blockchain identity are related but distinct.
- Authenticate strongly and authorize narrowly.
- Use individual identities for high-impact actions.
- Revoke quickly when roles change.

Implementation blueprint

- Map Keycloak subject ID to school staff ID and optional Fabric enrollment ID.
- Define role/attribute claims centrally.
- Document certificate issuance, renewal, revocation, and lost-token processes.
- Use short session lifetimes for privileged users.

Security and privacy controls

- MFA for privileged roles.
- No shared teacher/admin accounts.
- Separate CA admin from application admin.
- Protect recovery codes and token seeds.

Acceptance / verification

- ✓ Revoked user cannot submit.
- ✓ Changed role takes effect.
- ✓ Expired certificate handled cleanly.
- ✓ Shared-device logout removes session data.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

8.7 MFA on an offline LAN

MFA can remain local using TOTP generated from locally enrolled secrets, hardware tokens, smart cards, or FIDO2 keys. No Internet service is required once enrollment is complete.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Human identity, device identity, and blockchain identity are related but distinct.
- Authenticate strongly and authorize narrowly.
- Use individual identities for high-impact actions.
- Revoke quickly when roles change.

Implementation blueprint

- Map Keycloak subject ID to school staff ID and optional Fabric enrollment ID.
- Define role/attribute claims centrally.
- Document certificate issuance, renewal, revocation, and lost-token processes.
- Use short session lifetimes for privileged users.

Security and privacy controls

- MFA for privileged roles.
- No shared teacher/admin accounts.
- Separate CA admin from application admin.
- Protect recovery codes and token seeds.

Acceptance / verification

- ✓ Revoked user cannot submit.
- ✓ Changed role takes effect.
- ✓ Expired certificate handled cleanly.
- ✓ Shared-device logout removes session data.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

8.8 Joiner-mover-leaver process

Provision roles only after HR/registrar approval, update roles on job change, and revoke access immediately on departure. CRLs and application sessions must be updated promptly.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Human identity, device identity, and blockchain identity are related but distinct.
- Authenticate strongly and authorize narrowly.
- Use individual identities for high-impact actions.
- Revoke quickly when roles change.

Implementation blueprint

- Map Keycloak subject ID to school staff ID and optional Fabric enrollment ID.
- Define role/attribute claims centrally.
- Document certificate issuance, renewal, revocation, and lost-token processes.
- Use short session lifetimes for privileged users.

Security and privacy controls

- MFA for privileged roles.
- No shared teacher/admin accounts.
- Separate CA admin from application admin.
- Protect recovery codes and token seeds.

Acceptance / verification

- ✓ Revoked user cannot submit.
- ✓ Changed role takes effect.
- ✓ Expired certificate handled cleanly.
- ✓ Shared-device logout removes session data.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

8.9 Privileged access workstations

CA, Fabric admin, database admin, and backup administration should be performed from hardened management workstations on the management VLAN, not from classroom PCs.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Human identity, device identity, and blockchain identity are related but distinct.
- Authenticate strongly and authorize narrowly.
- Use individual identities for high-impact actions.
- Revoke quickly when roles change.

Implementation blueprint

- Map Keycloak subject ID to school staff ID and optional Fabric enrollment ID.
- Define role/attribute claims centrally.
- Document certificate issuance, renewal, revocation, and lost-token processes.
- Use short session lifetimes for privileged users.

Security and privacy controls

- MFA for privileged roles.
- No shared teacher/admin accounts.
- Separate CA admin from application admin.
- Protect recovery codes and token seeds.

Acceptance / verification

- ✓ Revoked user cannot submit.
- ✓ Changed role takes effect.
- ✓ Expired certificate handled cleanly.
- ✓ Shared-device logout removes session data.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

9.1 PII minimization rule

Do not put student names, birth dates, addresses, guardian details, health records, disciplinary notes, or raw assessment details directly into an immutable ledger. Store them off-chain and anchor hashes/IDs on-chain.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Minimize immutable personal data.
- Store sensitive details where they can be corrected and retained appropriately.
- Hashing is evidence, not anonymization of low-entropy fields.
- Access decisions must follow educational purpose.

Implementation blueprint

- Maintain a data inventory with field, owner, classification, location, retention, and legal basis.
- Use pseudonymous StudentUIDs on-chain.
- Encrypt databases, file stores, and backups.
- Log access to sensitive student records.

Security and privacy controls

- Never put private keys or passwords in the ledger.
- Avoid names/DOB/address in ledger keys/events.
- Limit exports and removable-media use.
- Review privileged read access.

Acceptance / verification

- ✓ Search ledger for prohibited PII.
- ✓ Verify role-based record access.
- ✓ Test deletion/retention off-chain.
- ✓ Test exported report redaction.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

9.2 Hybrid privacy model

PostgreSQL stores mutable student records; encrypted document storage holds files; Fabric stores pseudonymous IDs, hashes, approvals, version numbers, status, timestamps, and audit evidence.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Minimize immutable personal data.
- Store sensitive details where they can be corrected and retained appropriately.
- Hashing is evidence, not anonymization of low-entropy fields.
- Access decisions must follow educational purpose.

Implementation blueprint

- Maintain a data inventory with field, owner, classification, location, retention, and legal basis.
- Use pseudonymous StudentUIDs on-chain.
- Encrypt databases, file stores, and backups.
- Log access to sensitive student records.

Security and privacy controls

- Never put private keys or passwords in the ledger.
- Avoid names/DOB/address in ledger keys/events.
- Limit exports and removable-media use.
- Review privileged read access.

Acceptance / verification

- ✓ Search ledger for prohibited PII.
- ✓ Verify role-based record access.
- ✓ Test deletion/retention off-chain.
- ✓ Test exported report redaction.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

9.3 Education-record access controls

If the deployment is subject to FERPA, access should be restricted to legitimate educational interest, identity/authentication must be reliable, and disclosures/access should be auditable. [R12]

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Minimize immutable personal data.
- Store sensitive details where they can be corrected and retained appropriately.
- Hashing is evidence, not anonymization of low-entropy fields.
- Access decisions must follow educational purpose.

Implementation blueprint

- Maintain a data inventory with field, owner, classification, location, retention, and legal basis.
- Use pseudonymous StudentUIDs on-chain.
- Encrypt databases, file stores, and backups.
- Log access to sensitive student records.

Security and privacy controls

- Never put private keys or passwords in the ledger.
- Avoid names/DOB/address in ledger keys/events.
- Limit exports and removable-media use.
- Review privileged read access.

Acceptance / verification

- ✓ Search ledger for prohibited PII.
- ✓ Verify role-based record access.
- ✓ Test deletion/retention off-chain.
- ✓ Test exported report redaction.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

9.4 Data classification levels

Classify data as Public, Internal, Confidential Student, Restricted, and Cryptographic Secret. Map each class to storage, encryption, backup, retention, export, and logging rules.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Minimize immutable personal data.
- Store sensitive details where they can be corrected and retained appropriately.
- Hashing is evidence, not anonymization of low-entropy fields.
- Access decisions must follow educational purpose.

Implementation blueprint

- Maintain a data inventory with field, owner, classification, location, retention, and legal basis.
- Use pseudonymous StudentUIDs on-chain.
- Encrypt databases, file stores, and backups.
- Log access to sensitive student records.

Security and privacy controls

- Never put private keys or passwords in the ledger.
- Avoid names/DOB/address in ledger keys/events.
- Limit exports and removable-media use.
- Review privileged read access.

Acceptance / verification

- ✓ Search ledger for prohibited PII.
- ✓ Verify role-based record access.
- ✓ Test deletion/retention off-chain.
- ✓ Test exported report redaction.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

9.5 Encryption at rest

Encrypt server disks and backups. Use database encryption controls where available, but remember key management is the security boundary. Store master keys separately from backup media.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Minimize immutable personal data.
- Store sensitive details where they can be corrected and retained appropriately.
- Hashing is evidence, not anonymization of low-entropy fields.
- Access decisions must follow educational purpose.

Implementation blueprint

- Maintain a data inventory with field, owner, classification, location, retention, and legal basis.
- Use pseudonymous StudentUIDs on-chain.
- Encrypt databases, file stores, and backups.
- Log access to sensitive student records.

Security and privacy controls

- Never put private keys or passwords in the ledger.
- Avoid names/DOB/address in ledger keys/events.
- Limit exports and removable-media use.
- Review privileged read access.

Acceptance / verification

- ✓ Search ledger for prohibited PII.
- ✓ Verify role-based record access.
- ✓ Test deletion/retention off-chain.
- ✓ Test exported report redaction.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

9.6 Encryption in transit

Use TLS for browsers, APIs, database links, Fabric peer/orderer communication, CA enrollment, monitoring, and backup transfers even though traffic stays on the LAN.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Minimize immutable personal data.
- Store sensitive details where they can be corrected and retained appropriately.
- Hashing is evidence, not anonymization of low-entropy fields.
- Access decisions must follow educational purpose.

Implementation blueprint

- Maintain a data inventory with field, owner, classification, location, retention, and legal basis.
- Use pseudonymous StudentUIDs on-chain.
- Encrypt databases, file stores, and backups.
- Log access to sensitive student records.

Security and privacy controls

- Never put private keys or passwords in the ledger.
- Avoid names/DOB/address in ledger keys/events.
- Limit exports and removable-media use.
- Review privileged read access.

Acceptance / verification

- ✓ Search ledger for prohibited PII.
- ✓ Verify role-based record access.
- ✓ Test deletion/retention off-chain.
- ✓ Test exported report redaction.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

9.7 Retention and deletion

The ledger should contain only records with a defensible long-term retention purpose. Off-chain data can follow school retention/deletion rules; retained ledger hashes should not reconstruct deleted PII.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Minimize immutable personal data.
- Store sensitive details where they can be corrected and retained appropriately.
- Hashing is evidence, not anonymization of low-entropy fields.
- Access decisions must follow educational purpose.

Implementation blueprint

- Maintain a data inventory with field, owner, classification, location, retention, and legal basis.
- Use pseudonymous StudentUIDs on-chain.
- Encrypt databases, file stores, and backups.
- Log access to sensitive student records.

Security and privacy controls

- Never put private keys or passwords in the ledger.
- Avoid names/DOB/address in ledger keys/events.
- Limit exports and removable-media use.
- Review privileged read access.

Acceptance / verification

- ✓ Search ledger for prohibited PII.
- ✓ Verify role-based record access.
- ✓ Test deletion/retention off-chain.
- ✓ Test exported report redaction.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

9.8 Private Data Collections

Use Fabric PDCs only when multiple organizations must transact on sensitive shared state and the data still belongs in the ledger model. PDCs are not a substitute for general-purpose student-record storage. [R7]

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Minimize immutable personal data.
- Store sensitive details where they can be corrected and retained appropriately.
- Hashing is evidence, not anonymization of low-entropy fields.
- Access decisions must follow educational purpose.

Implementation blueprint

- Maintain a data inventory with field, owner, classification, location, retention, and legal basis.
- Use pseudonymous StudentUIDs on-chain.
- Encrypt databases, file stores, and backups.
- Log access to sensitive student records.

Security and privacy controls

- Never put private keys or passwords in the ledger.
- Avoid names/DOB/address in ledger keys/events.
- Limit exports and removable-media use.
- Review privileged read access.

Acceptance / verification

- ✓ Search ledger for prohibited PII.
- ✓ Verify role-based record access.
- ✓ Test deletion/retention off-chain.
- ✓ Test exported report redaction.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

9.9 Privacy impact assessment

Before adding a new module, document the purpose, data fields, users, access rules, retention, on-chain content, exports, failure consequences, and correction process.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Minimize immutable personal data.
- Store sensitive details where they can be corrected and retained appropriately.
- Hashing is evidence, not anonymization of low-entropy fields.
- Access decisions must follow educational purpose.

Implementation blueprint

- Maintain a data inventory with field, owner, classification, location, retention, and legal basis.
- Use pseudonymous StudentUIDs on-chain.
- Encrypt databases, file stores, and backups.
- Log access to sensitive student records.

Security and privacy controls

- Never put private keys or passwords in the ledger.
- Avoid names/DOB/address in ledger keys/events.
- Limit exports and removable-media use.
- Review privileged read access.

Acceptance / verification

- ✓ Search ledger for prohibited PII.
- ✓ Verify role-based record access.
- ✓ Test deletion/retention off-chain.
- ✓ Test exported report redaction.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

10.1 Stable identifiers

Use opaque UUIDs or school-issued internal IDs as ledger keys. Avoid student names, email addresses, national IDs, or classroom labels in composite keys.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Canonicalize before hashing.
- Version instead of overwriting evidence.
- Use idempotency for retry-safe writes.
- Keep analytics indexes off-chain.

Implementation blueprint

- Define JSON schemas and version them.
- Use UTC timestamps plus local school-date fields where needed.
- Store correlation IDs across UI/API/database/ledger logs.
- Document reconciliation rules.

Security and privacy controls

- Validate hash length/format.
- Reject stale expectedVersion.
- Prevent duplicate request IDs.
- Do not accept client-supplied actor roles without server verification.

Acceptance / verification

- ✓ Same canonical record produces same hash.
- ✓ Changed field changes hash.
- ✓ Duplicate request is idempotent.
- ✓ Version conflict is rejected.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

10.2 Canonical JSON hashing

Normalize field order, number/date formats, Unicode, and omitted/null values before hashing. If two systems hash semantically identical records differently, verification becomes unreliable.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Canonicalize before hashing.
- Version instead of overwriting evidence.
- Use idempotency for retry-safe writes.
- Keep analytics indexes off-chain.

Implementation blueprint

- Define JSON schemas and version them.
- Use UTC timestamps plus local school-date fields where needed.
- Store correlation IDs across UI/API/database/ledger logs.
- Document reconciliation rules.

Security and privacy controls

- Validate hash length/format.
- Reject stale expectedVersion.
- Prevent duplicate request IDs.
- Do not accept client-supplied actor roles without server verification.

Acceptance / verification

- ✓ Same canonical record produces same hash.
- ✓ Changed field changes hash.
- ✓ Duplicate request is idempotent.
- ✓ Version conflict is rejected.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

10.3 Hash algorithm and domain separation

Use a modern collision-resistant hash such as SHA-256 and prefix each hash input with a record type/version so a transcript hash cannot be confused with an attendance hash.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Canonicalize before hashing.
- Version instead of overwriting evidence.
- Use idempotency for retry-safe writes.
- Keep analytics indexes off-chain.

Implementation blueprint

- Define JSON schemas and version them.
- Use UTC timestamps plus local school-date fields where needed.
- Store correlation IDs across UI/API/database/ledger logs.
- Document reconciliation rules.

Security and privacy controls

- Validate hash length/format.
- Reject stale expectedVersion.
- Prevent duplicate request IDs.
- Do not accept client-supplied actor roles without server verification.

Acceptance / verification

- ✓ Same canonical record produces same hash.
- ✓ Changed field changes hash.
- ✓ Duplicate request is idempotent.
- ✓ Version conflict is rejected.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

10.4 Versioned records

Every off-chain record anchored on-chain should carry version, contentHash, previousHash or priorVersion reference, actor, timestamp, and reason code for correction.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Canonicalize before hashing.
- Version instead of overwriting evidence.
- Use idempotency for retry-safe writes.
- Keep analytics indexes off-chain.

Implementation blueprint

- Define JSON schemas and version them.
- Use UTC timestamps plus local school-date fields where needed.
- Store correlation IDs across UI/API/database/ledger logs.
- Document reconciliation rules.

Security and privacy controls

- Validate hash length/format.
- Reject stale expectedVersion.
- Prevent duplicate request IDs.
- Do not accept client-supplied actor roles without server verification.

Acceptance / verification

- ✓ Same canonical record produces same hash.
- ✓ Changed field changes hash.
- ✓ Duplicate request is idempotent.
- ✓ Version conflict is rejected.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

10.5 Create versus amend

Never overwrite history conceptually. A correction creates a new version and marks the prior version superseded. The application shows current state while the ledger preserves the full sequence.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Canonicalize before hashing.
- Version instead of overwriting evidence.
- Use idempotency for retry-safe writes.
- Keep analytics indexes off-chain.

Implementation blueprint

- Define JSON schemas and version them.
- Use UTC timestamps plus local school-date fields where needed.
- Store correlation IDs across UI/API/database/ledger logs.
- Document reconciliation rules.

Security and privacy controls

- Validate hash length/format.
- Reject stale expectedVersion.
- Prevent duplicate request IDs.
- Do not accept client-supplied actor roles without server verification.

Acceptance / verification

- ✓ Same canonical record produces same hash.
- ✓ Changed field changes hash.
- ✓ Duplicate request is idempotent.
- ✓ Version conflict is rejected.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

10.6 Idempotency keys

Every client mutation should include an idempotency/request ID. Chaincode rejects duplicate request IDs so retries after network timeouts do not create duplicate attendance or certificate events.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Canonicalize before hashing.
- Version instead of overwriting evidence.
- Use idempotency for retry-safe writes.
- Keep analytics indexes off-chain.

Implementation blueprint

- Define JSON schemas and version them.
- Use UTC timestamps plus local school-date fields where needed.
- Store correlation IDs across UI/API/database/ledger logs.
- Document reconciliation rules.

Security and privacy controls

- Validate hash length/format.
- Reject stale expectedVersion.
- Prevent duplicate request IDs.
- Do not accept client-supplied actor roles without server verification.

Acceptance / verification

- ✓ Same canonical record produces same hash.
- ✓ Changed field changes hash.
- ✓ Duplicate request is idempotent.
- ✓ Version conflict is rejected.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

10.7 Composite keys and indexes

Choose ledger keys for deterministic retrieval, but keep complex reporting indexes in PostgreSQL. Fabric world state is not a replacement for an analytics database.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Canonicalize before hashing.
- Version instead of overwriting evidence.
- Use idempotency for retry-safe writes.
- Keep analytics indexes off-chain.

Implementation blueprint

- Define JSON schemas and version them.
- Use UTC timestamps plus local school-date fields where needed.
- Store correlation IDs across UI/API/database/ledger logs.
- Document reconciliation rules.

Security and privacy controls

- Validate hash length/format.
- Reject stale expectedVersion.
- Prevent duplicate request IDs.
- Do not accept client-supplied actor roles without server verification.

Acceptance / verification

- ✓ Same canonical record produces same hash.
- ✓ Changed field changes hash.
- ✓ Duplicate request is idempotent.
- ✓ Version conflict is rejected.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

10.8 On-chain event envelope

Standardize event payload fields: schemaVersion, eventType, entityId, actorId, organization, transactionId, timestamp, contentHash, correlationId, and minimal non-PII status.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Canonicalize before hashing.
- Version instead of overwriting evidence.
- Use idempotency for retry-safe writes.
- Keep analytics indexes off-chain.

Implementation blueprint

- Define JSON schemas and version them.
- Use UTC timestamps plus local school-date fields where needed.
- Store correlation IDs across UI/API/database/ledger logs.
- Document reconciliation rules.

Security and privacy controls

- Validate hash length/format.
- Reject stale expectedVersion.
- Prevent duplicate request IDs.
- Do not accept client-supplied actor roles without server verification.

Acceptance / verification

- ✓ Same canonical record produces same hash.
- ✓ Changed field changes hash.
- ✓ Duplicate request is idempotent.
- ✓ Version conflict is rejected.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

10.9 Audit provenance

Every ledger write should capture who requested it, which role/organization authorized it, which source record was hashed, and which approval workflow produced the final action.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Canonicalize before hashing.
- Version instead of overwriting evidence.
- Use idempotency for retry-safe writes.
- Keep analytics indexes off-chain.

Implementation blueprint

- Define JSON schemas and version them.
- Use UTC timestamps plus local school-date fields where needed.
- Store correlation IDs across UI/API/database/ledger logs.
- Document reconciliation rules.

Security and privacy controls

- Validate hash length/format.
- Reject stale expectedVersion.
- Prevent duplicate request IDs.
- Do not accept client-supplied actor roles without server verification.

Acceptance / verification

- ✓ Same canonical record produces same hash.
- ✓ Changed field changes hash.
- ✓ Duplicate request is idempotent.
- ✓ Version conflict is rejected.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

11.1 Student application intake

Collect admissions data in the off-chain application database. Before approval, records remain mutable drafts and are not written to the blockchain.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

11.2 Admission approval transaction

When approved, create a pseudonymous StudentUID, compute a canonical profile hash, and submit an EnrollmentApproved ledger transaction with program, cohort code, approval references, and version.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

11.3 Duplicate detection

Use off-chain matching rules for duplicate applicants; do not expose matching identifiers to the ledger. The ledger can store a duplicate-resolution decision ID and evidence hash.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

11.4 Student status lifecycle

Model statuses such as APPLICANT, ACTIVE, SUSPENDED, GRADUATED, WITHDRAWN, and ARCHIVED. Restrict transitions and require reason codes/approvals for exceptional changes.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

11.5 Guardian and contact data

Keep guardian names, addresses, phone numbers, emails, and custody information off-chain with strict access control. Only hash a versioned contact profile when audit evidence is needed.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

11.6 Transfer-in workflow

For students arriving from another school, validate documents, store scans off-chain, record verification evidence, and anchor an admission/credential verification transaction.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

11.7 Transfer-out workflow

Generate an official transcript/certificate package, record the issuance hash and destination organization code, then change status according to school policy.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

11.8 Student profile correction

Allow authorized registrar staff to correct off-chain information. Submit a profile-version hash with reason and approval if the changed field is within the audit scope.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

11.9 Admissions acceptance tests

Test duplicate requests, invalid status transitions, unauthorized staff, tampered profile hashes, resubmission after timeout, bulk admission import, and rollback after application failure.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

12.1 Attendance data model

Represent attendance with StudentUID, class/session ID, school date, status code, recorder, source, revision, and record hash. Keep explanatory notes off-chain.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

12.2 Teacher workflow

Teacher opens a local classroom page, marks attendance, reviews a draft, and submits the class roster. The backend validates roster membership before producing ledger writes/events.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

12.3 Batch attendance design

Do not necessarily create one expensive governance transaction per click. Use a class-session batch with a Merkle/root or roster hash plus off-chain rows, or compact per-student events depending on audit requirements.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

12.4 Late corrections

Corrections create a revision rather than deleting the prior record. Require reason code and, after a configurable cutoff, department-head approval.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

12.5 Offline classroom interruption

If the LAN is temporarily unavailable, store signed local drafts on the teacher device with sequence/time metadata and synchronize when the application becomes reachable. Reject duplicates by idempotency key.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

12.6 Timetable authority

The timetable remains an operational database. Anchor only published schedule versions or hashes when proof of the official timetable is needed.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

12.7 Attendance reporting

Generate daily/monthly summaries from PostgreSQL. Use ledger transaction IDs and hashes as evidence links, not as the analytics engine.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

12.8 Fraud controls

Prevent teachers from editing classes they do not teach, changing attendance outside allowed windows, impersonating another teacher, or mass-editing without secondary approval.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

12.9 Attendance acceptance tests

Test class-size batches, double submission, roster mismatch, teacher substitution, correction windows, device clock drift, server restart, and ledger/database reconciliation.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

13.1 Assessment lifecycle

Model assessments as DRAFT, MODERATED, APPROVED, PUBLISHED, and CORRECTED. Draft marks stay off-chain; blockchain evidence begins at controlled approval points.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

13.2 Grade data privacy

Exact grades are education records and should generally remain off-chain. Anchor a hash of the approved grade sheet plus course/term identifiers and approval signatures.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

13.3 Teacher mark entry

Teachers enter marks into the local application. Validation enforces score ranges, enrollment, grading scheme, and subject ownership before moderation.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

13.4 Moderation and approval

A second role reviews outliers and completeness. When the grade sheet is approved, compute the canonical hash and submit a GradeSheetPublished transaction requiring AcademicOrg and RegistrarOrg endorsement.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

13.5 Grade correction

Corrections require a reason, prior hash/version, authorizer, and new grade-sheet hash. The original published record remains in history.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

13.6 GPA computation

Compute GPA and rankings off-chain from approved grade versions. Store algorithm version and report hash so a later transcript can prove which rules were used.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

13.7 Transcript generation

Render transcripts locally from current approved records, include a unique transcript ID and content hash, sign the PDF with a school key, and anchor issuance metadata on Fabric.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

13.8 Disputed grade workflow

Open a case off-chain, freeze further publication if necessary, record approvals and final resolution hash, then publish a correction transaction if the official grade changes.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

13.9 Exam module acceptance tests

Test missing marks, unauthorized publication, conflicting approvals, grade correction, hash mismatch, reprint of historical transcript, and reconciliation between database versions and ledger anchors.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

14.1 Credential data model

Store credential ID, StudentUID or pseudonymous holder ID, credential type, program, issue date, content hash, issuer key ID, status, and revocation information.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

14.2 Certificate generation

Generate the certificate PDF locally, canonicalize credential metadata, hash it, digitally sign the credential package, and submit a CredentialIssued ledger transaction.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

14.3 QR code design

A QR can encode credential ID plus signed compact metadata. A LAN verifier resolves the current ledger status; a standalone offline verifier can validate the signature against a bundled school public key.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

14.4 Offline external verification

Pure offline verification proves the signature and content integrity but may not know the latest revocation state. Provide a dated signed revocation list or require periodic connection to the school LAN for freshest status.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

14.5 Revocation workflow

Revocation never deletes issuance history. Submit `CredentialRevoked` with reason category, authority, timestamp, and prior credential ID; verification surfaces revoked status prominently.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

14.6 Reissue workflow

A reissued certificate gets a new credential/version ID and references the prior issuance. The old credential may remain valid, superseded, or revoked according to policy.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

14.7 Digital signature keys

Keep credential-signing keys in a protected keystore/HSM where feasible. Separate document-signing keys from Fabric CA keys and rotate them under a documented policy.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

14.8 Verification portal

Host a read-only local web page that accepts credential ID/QR input, queries the current ledger via a restricted Gateway identity, and displays status plus a minimal non-sensitive summary.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

14.9 Credential acceptance tests

Test tampered PDF, changed QR data, revoked credential, old signing key, duplicate issuance, reissue, missing ledger node, and offline verifier with stale revocation data.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

15.1 Library loan audit

Keep current catalog/loan operations in a database. Anchor exceptional or high-value asset handovers, write-offs, and disputed return decisions if tamper-evident evidence is beneficial.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

15.2 Textbook/device custody

For laptops/tablets or expensive equipment, record custody transfers with asset ID, custodian pseudonymous ID, condition hash, timestamp, and signatures/approvals.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

15.3 School fee status

Avoid storing bank/account/payment-card data on-chain. If the school tracks fees, store receipt/reference hashes and status transitions, while financial details remain in the accounting system.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

15.4 Scholarships and waivers

Store authorization evidence for scholarship/waiver decisions without exposing sensitive socioeconomic details. Chain data should record decision ID, policy code, approving roles, and evidence hash.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

15.5 Staff credential records

The same credential model can issue staff training or authorization certificates. Keep HR personal data outside the ledger.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

15.6 Inventory audit

Use the ledger for custody/approval events on high-value assets; use the database for counts, depreciation, maintenance schedules, and search.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

15.7 Disciplinary records

Do not place disciplinary narrative or allegations on an immutable ledger. At most, anchor final decision evidence when legally and operationally justified.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

15.8 Health and counseling data

Exclude medical, counseling, and highly sensitive welfare records from the blockchain design unless a specialist legal/privacy review explicitly requires a narrow audit anchor.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

15.9 Optional module decision checklist

Before adding a module, ask whether multiple authorities must approve, whether tamper evidence matters, whether data can remain pseudonymous, whether correction/deletion is needed, and whether the operational database alone is sufficient.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Drafts remain mutable off-chain; blockchain evidence begins at an approved business boundary.
- Every correction creates provenance.
- High-impact actions require secondary approval.
- Reports come from the operational store and are anchored when needed.

Implementation blueprint

- Define state machine, roles, fields, validation, approvals, on-chain evidence, off-chain tables, and events.
- Use stable pseudonymous identifiers.
- Attach reason codes to exceptions/corrections.
- Provide a human-readable history view.

Security and privacy controls

- Enforce row/object ownership and role scope.
- Prevent back-dated changes without explicit exception authority.
- Keep narrative/sensitive notes off-chain.
- Audit bulk changes.

Acceptance / verification

- ✓ Happy path.
- ✓ Unauthorized actor.
- ✓ Duplicate/retry.
- ✓ Correction/reversal.
- ✓ Tampered hash.
- ✓ Service restart mid-workflow.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

16.1 Backend service architecture

Use a local API layer between browsers and Fabric. The API authenticates users, enforces application policy, validates input, queries PostgreSQL, submits Fabric transactions through Gateway, and records correlation IDs.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Browsers should not talk directly to databases or privileged Fabric services.
- Business authorization belongs server-side.
- Use correlation IDs and idempotency.
- Index events for reporting instead of querying blocks for every screen.

Implementation blueprint

- Separate auth, domain service, Fabric adapter, repository, event indexer, report service, and audit logger.
- Use a transaction outbox/job table for reliable asynchronous work.
- Health endpoints must test dependencies separately.
- Document API schemas and error codes.

Security and privacy controls

- Validate all input.
- Use least-privilege DB and Fabric identities.
- Secure cookies and CSRF defenses.
- Rate-limit sensitive endpoints.

Acceptance / verification

- ✓ API auth matrix.
- ✓ Fabric timeout/retry.
- ✓ DB rollback.
- ✓ Event replay.
- ✓ Duplicate request.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

16.2 Fabric Gateway client

Use the current Fabric Gateway client API for Go, Node.js, or Java. Avoid deprecated legacy SDKs for new Fabric v3 applications. [R4]

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Browsers should not talk directly to databases or privileged Fabric services.
- Business authorization belongs server-side.
- Use correlation IDs and idempotency.
- Index events for reporting instead of querying blocks for every screen.

Implementation blueprint

- Separate auth, domain service, Fabric adapter, repository, event indexer, report service, and audit logger.
- Use a transaction outbox/job table for reliable asynchronous work.
- Health endpoints must test dependencies separately.
- Document API schemas and error codes.

Security and privacy controls

- Validate all input.
- Use least-privilege DB and Fabric identities.
- Secure cookies and CSRF defenses.
- Rate-limit sensitive endpoints.

Acceptance / verification

- ✓ API auth matrix.
- ✓ Fabric timeout/retry.
- ✓ DB rollback.
- ✓ Event replay.
- ✓ Duplicate request.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

16.3 Transaction request envelope

Standardize API mutation requests with requestId, actorId, role, entityId, expectedVersion, payloadHash, reasonCode, clientTimestamp, and optional approval references.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Browsers should not talk directly to databases or privileged Fabric services.
- Business authorization belongs server-side.
- Use correlation IDs and idempotency.
- Index events for reporting instead of querying blocks for every screen.

Implementation blueprint

- Separate auth, domain service, Fabric adapter, repository, event indexer, report service, and audit logger.
- Use a transaction outbox/job table for reliable asynchronous work.
- Health endpoints must test dependencies separately.
- Document API schemas and error codes.

Security and privacy controls

- Validate all input.
- Use least-privilege DB and Fabric identities.
- Secure cookies and CSRF defenses.
- Rate-limit sensitive endpoints.

Acceptance / verification

- ✓ API auth matrix.
- ✓ Fabric timeout/retry.
- ✓ DB rollback.
- ✓ Event replay.
- ✓ Duplicate request.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

16.4 Synchronous versus asynchronous

For high-value writes, return after commit status is known. For bulk imports, queue locally, process asynchronously, and expose job status while preserving idempotency.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Browsers should not talk directly to databases or privileged Fabric services.
- Business authorization belongs server-side.
- Use correlation IDs and idempotency.
- Index events for reporting instead of querying blocks for every screen.

Implementation blueprint

- Separate auth, domain service, Fabric adapter, repository, event indexer, report service, and audit logger.
- Use a transaction outbox/job table for reliable asynchronous work.
- Health endpoints must test dependencies separately.
- Document API schemas and error codes.

Security and privacy controls

- Validate all input.
- Use least-privilege DB and Fabric identities.
- Secure cookies and CSRF defenses.
- Rate-limit sensitive endpoints.

Acceptance / verification

- ✓ API auth matrix.
- ✓ Fabric timeout/retry.
- ✓ DB rollback.
- ✓ Event replay.
- ✓ Duplicate request.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

16.5 Event indexer

Consume Fabric block/chaincode events and write a read model to PostgreSQL for dashboards, reconciliation, notifications, and reports. Store block number and transaction ID as checkpoints.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Browsers should not talk directly to databases or privileged Fabric services.
- Business authorization belongs server-side.
- Use correlation IDs and idempotency.
- Index events for reporting instead of querying blocks for every screen.

Implementation blueprint

- Separate auth, domain service, Fabric adapter, repository, event indexer, report service, and audit logger.
- Use a transaction outbox/job table for reliable asynchronous work.
- Health endpoints must test dependencies separately.
- Document API schemas and error codes.

Security and privacy controls

- Validate all input.
- Use least-privilege DB and Fabric identities.
- Secure cookies and CSRF defenses.
- Rate-limit sensitive endpoints.

Acceptance / verification

- ✓ API auth matrix.
- ✓ Fabric timeout/retry.
- ✓ DB rollback.
- ✓ Event replay.
- ✓ Duplicate request.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

16.6 Reconciliation service

Periodically compare database anchors against ledger state and detect missing hashes, wrong versions, failed events, or unexpected ledger-only transactions.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Browsers should not talk directly to databases or privileged Fabric services.
- Business authorization belongs server-side.
- Use correlation IDs and idempotency.
- Index events for reporting instead of querying blocks for every screen.

Implementation blueprint

- Separate auth, domain service, Fabric adapter, repository, event indexer, report service, and audit logger.
- Use a transaction outbox/job table for reliable asynchronous work.
- Health endpoints must test dependencies separately.
- Document API schemas and error codes.

Security and privacy controls

- Validate all input.
- Use least-privilege DB and Fabric identities.
- Secure cookies and CSRF defenses.
- Rate-limit sensitive endpoints.

Acceptance / verification

- ✓ API auth matrix.
- ✓ Fabric timeout/retry.
- ✓ DB rollback.
- ✓ Event replay.
- ✓ Duplicate request.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

16.7 REST API design

Expose versioned endpoints such as `/api/v1/students`, `/attendance`, `/grades/publish`, `/credentials`, `/audit`, and `/verify`. Keep Fabric-specific internals out of browser code.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Browsers should not talk directly to databases or privileged Fabric services.
- Business authorization belongs server-side.
- Use correlation IDs and idempotency.
- Index events for reporting instead of querying blocks for every screen.

Implementation blueprint

- Separate auth, domain service, Fabric adapter, repository, event indexer, report service, and audit logger.
- Use a transaction outbox/job table for reliable asynchronous work.
- Health endpoints must test dependencies separately.
- Document API schemas and error codes.

Security and privacy controls

- Validate all input.
- Use least-privilege DB and Fabric identities.
- Secure cookies and CSRF defenses.
- Rate-limit sensitive endpoints.

Acceptance / verification

- ✓ API auth matrix.
- ✓ Fabric timeout/retry.
- ✓ DB rollback.
- ✓ Event replay.
- ✓ Duplicate request.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

16.8 Integration with existing SIS

If a school already has an SIS, treat it as an upstream/downstream system. Use scheduled local exports or APIs, canonical mapping, replay protection, and reconciliation rather than replacing everything at once.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Browsers should not talk directly to databases or privileged Fabric services.
- Business authorization belongs server-side.
- Use correlation IDs and idempotency.
- Index events for reporting instead of querying blocks for every screen.

Implementation blueprint

- Separate auth, domain service, Fabric adapter, repository, event indexer, report service, and audit logger.
- Use a transaction outbox/job table for reliable asynchronous work.
- Health endpoints must test dependencies separately.
- Document API schemas and error codes.

Security and privacy controls

- Validate all input.
- Use least-privilege DB and Fabric identities.
- Secure cookies and CSRF defenses.
- Rate-limit sensitive endpoints.

Acceptance / verification

- ✓ API auth matrix.
- ✓ Fabric timeout/retry.
- ✓ DB rollback.
- ✓ Event replay.
- ✓ Duplicate request.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

16.9 API acceptance tests

Test authentication, authorization, validation, idempotency, timeout retry, Gateway unavailable, database unavailable, partial failure, malformed hash, and event-indexer restart.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Browsers should not talk directly to databases or privileged Fabric services.
- Business authorization belongs server-side.
- Use correlation IDs and idempotency.
- Index events for reporting instead of querying blocks for every screen.

Implementation blueprint

- Separate auth, domain service, Fabric adapter, repository, event indexer, report service, and audit logger.
- Use a transaction outbox/job table for reliable asynchronous work.
- Health endpoints must test dependencies separately.
- Document API schemas and error codes.

Security and privacy controls

- Validate all input.
- Use least-privilege DB and Fabric identities.
- Secure cookies and CSRF defenses.
- Rate-limit sensitive endpoints.

Acceptance / verification

- ✓ API auth matrix.
- ✓ Fabric timeout/retry.
- ✓ DB rollback.
- ✓ Event replay.
- ✓ Duplicate request.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

17.1 Local web application

Host all JavaScript, CSS, fonts, icons, and assets locally. No page should call public CDNs, analytics, maps, captcha, or external identity providers.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Make blockchain invisible unless its evidence is useful to the user.
- No external web dependencies.
- Explain transaction states.
- Design for staff speed and accessibility.

Implementation blueprint

- Bundle all assets locally.
- Use role-specific navigation.
- Cache only non-sensitive reference data.
- Provide print/export workflows with clear provenance.

Security and privacy controls

- No secrets in browser code.
- Short sessions on shared devices.
- Disable autocomplete for high-risk admin actions where appropriate.
- Use CSP and secure headers.

Acceptance / verification

- ✓ Keyboard-only navigation.
- ✓ Session timeout.
- ✓ Wrong-role deep link.
- ✓ LAN disconnect/reconnect.
- ✓ Large roster rendering.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

17.2 Role-specific dashboards

Registrar sees admissions/credentials, teachers see assigned classes, exam officers see moderation queues, principal sees approvals, auditors see read-only evidence, and students see only their own permitted records.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Make blockchain invisible unless its evidence is useful to the user.
- No external web dependencies.
- Explain transaction states.
- Design for staff speed and accessibility.

Implementation blueprint

- Bundle all assets locally.
- Use role-specific navigation.
- Cache only non-sensitive reference data.
- Provide print/export workflows with clear provenance.

Security and privacy controls

- No secrets in browser code.
- Short sessions on shared devices.
- Disable autocomplete for high-risk admin actions where appropriate.
- Use CSP and secure headers.

Acceptance / verification

- ✓ Keyboard-only navigation.
- ✓ Session timeout.
- ✓ Wrong-role deep link.
- ✓ LAN disconnect/reconnect.
- ✓ Large roster rendering.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

17.3 Transaction UX

Show states such as Draft, Validating, Awaiting approval, Submitted to ledger, Committed, Failed, and Requires correction. Do not display a spinner without explaining whether the transaction is actually final.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Make blockchain invisible unless its evidence is useful to the user.
- No external web dependencies.
- Explain transaction states.
- Design for staff speed and accessibility.

Implementation blueprint

- Bundle all assets locally.
- Use role-specific navigation.
- Cache only non-sensitive reference data.
- Provide print/export workflows with clear provenance.

Security and privacy controls

- No secrets in browser code.
- Short sessions on shared devices.
- Disable autocomplete for high-risk admin actions where appropriate.
- Use CSP and secure headers.

Acceptance / verification

- ✓ Keyboard-only navigation.
- ✓ Session timeout.
- ✓ Wrong-role deep link.
- ✓ LAN disconnect/reconnect.
- ✓ Large roster rendering.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

17.4 Audit evidence links

Every committed record should offer an evidence drawer with transaction ID, ledger timestamp, version, content hash, approvers, and human-readable reason without exposing cryptographic complexity unnecessarily.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Make blockchain invisible unless its evidence is useful to the user.
- No external web dependencies.
- Explain transaction states.
- Design for staff speed and accessibility.

Implementation blueprint

- Bundle all assets locally.
- Use role-specific navigation.
- Cache only non-sensitive reference data.
- Provide print/export workflows with clear provenance.

Security and privacy controls

- No secrets in browser code.
- Short sessions on shared devices.
- Disable autocomplete for high-risk admin actions where appropriate.
- Use CSP and secure headers.

Acceptance / verification

- ✓ Keyboard-only navigation.
- ✓ Session timeout.
- ✓ Wrong-role deep link.
- ✓ LAN disconnect/reconnect.
- ✓ Large roster rendering.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

17.5 Bulk workflows

Attendance, enrollment import, and grade publication need bulk review screens with validation summaries and partial-failure handling. Never force staff to click hundreds of blockchain actions individually.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Make blockchain invisible unless its evidence is useful to the user.
- No external web dependencies.
- Explain transaction states.
- Design for staff speed and accessibility.

Implementation blueprint

- Bundle all assets locally.
- Use role-specific navigation.
- Cache only non-sensitive reference data.
- Provide print/export workflows with clear provenance.

Security and privacy controls

- No secrets in browser code.
- Short sessions on shared devices.
- Disable autocomplete for high-risk admin actions where appropriate.
- Use CSP and secure headers.

Acceptance / verification

- ✓ Keyboard-only navigation.
- ✓ Session timeout.
- ✓ Wrong-role deep link.
- ✓ LAN disconnect/reconnect.
- ✓ Large roster rendering.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

17.6 Report generation

Generate PDFs/CSVs locally from approved database snapshots. Embed report ID, generation time, schema/algorithm version, and optional ledger anchor hash.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Make blockchain invisible unless its evidence is useful to the user.
- No external web dependencies.
- Explain transaction states.
- Design for staff speed and accessibility.

Implementation blueprint

- Bundle all assets locally.
- Use role-specific navigation.
- Cache only non-sensitive reference data.
- Provide print/export workflows with clear provenance.

Security and privacy controls

- No secrets in browser code.
- Short sessions on shared devices.
- Disable autocomplete for high-risk admin actions where appropriate.
- Use CSP and secure headers.

Acceptance / verification

- ✓ Keyboard-only navigation.
- ✓ Session timeout.
- ✓ Wrong-role deep link.
- ✓ LAN disconnect/reconnect.
- ✓ Large roster rendering.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

17.7 Accessibility

Use keyboard navigation, clear labels, sufficient contrast, screen-reader-friendly form structure, understandable errors, and printable alternatives. Blockchain features should not reduce educational accessibility.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Make blockchain invisible unless its evidence is useful to the user.
- No external web dependencies.
- Explain transaction states.
- Design for staff speed and accessibility.

Implementation blueprint

- Bundle all assets locally.
- Use role-specific navigation.
- Cache only non-sensitive reference data.
- Provide print/export workflows with clear provenance.

Security and privacy controls

- No secrets in browser code.
- Short sessions on shared devices.
- Disable autocomplete for high-risk admin actions where appropriate.
- Use CSP and secure headers.

Acceptance / verification

- ✓ Keyboard-only navigation.
- ✓ Session timeout.
- ✓ Wrong-role deep link.
- ✓ LAN disconnect/reconnect.
- ✓ Large roster rendering.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

17.8 Kiosk and shared devices

Use short sessions, automatic logout, no credential caching, privacy screens where appropriate, and separate student/staff kiosk profiles. Never install admin certificates on shared devices.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Make blockchain invisible unless its evidence is useful to the user.
- No external web dependencies.
- Explain transaction states.
- Design for staff speed and accessibility.

Implementation blueprint

- Bundle all assets locally.
- Use role-specific navigation.
- Cache only non-sensitive reference data.
- Provide print/export workflows with clear provenance.

Security and privacy controls

- No secrets in browser code.
- Short sessions on shared devices.
- Disable autocomplete for high-risk admin actions where appropriate.
- Use CSP and secure headers.

Acceptance / verification

- ✓ Keyboard-only navigation.
- ✓ Session timeout.
- ✓ Wrong-role deep link.
- ✓ LAN disconnect/reconnect.
- ✓ Large roster rendering.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

17.9 Frontend acceptance tests

Test every role, wrong-role access, stale session, wrong student record, network interruption, resubmission, browser refresh during commit, printing, large class rosters, and accessibility workflows.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Make blockchain invisible unless its evidence is useful to the user.
- No external web dependencies.
- Explain transaction states.
- Design for staff speed and accessibility.

Implementation blueprint

- Bundle all assets locally.
- Use role-specific navigation.
- Cache only non-sensitive reference data.
- Provide print/export workflows with clear provenance.

Security and privacy controls

- No secrets in browser code.
- Short sessions on shared devices.
- Disable autocomplete for high-risk admin actions where appropriate.
- Use CSP and secure headers.

Acceptance / verification

- ✓ Keyboard-only navigation.
- ✓ Session timeout.
- ✓ Wrong-role deep link.
- ✓ LAN disconnect/reconnect.
- ✓ Large roster rendering.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

18.1 Threat model

Model malicious insider, compromised teacher device, stolen admin credentials, rogue student device, ransomware, database tampering, CA key theft, orderer compromise, backup theft, supply-chain malware, and physical server theft.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Assume credentials and endpoints can be compromised.
- Layer preventive, detective, and recovery controls.
- Protect resources rather than trusting network location.
- Backups are part of security, not only operations.

Implementation blueprint

- Maintain threat model, asset inventory, dependency/SBOM inventory, privileged-account list, and security baseline.
- Centralize logs and time sync.
- Use offline patch staging and malware scanning.
- Run incident exercises.

Security and privacy controls

- MFA, segmentation, least privilege, key protection, immutable/offline backups, secure updates.
- No direct Internet on production servers.
- No shared privileged passwords.
- No silent security exceptions.

Acceptance / verification

- ✓ Credential revocation.
- ✓ Lateral movement denial.
- ✓ Restore after ransomware simulation.
- ✓ Vulnerability scan and remediation verification.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

18.2 Zero-trust principle on a LAN

Do not grant trust merely because a device is on school Ethernet or Wi-Fi. Authenticate users/devices and authorize each resource request. NIST explicitly notes that network location should not create implicit trust. [R13]

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Assume credentials and endpoints can be compromised.
- Layer preventive, detective, and recovery controls.
- Protect resources rather than trusting network location.
- Backups are part of security, not only operations.

Implementation blueprint

- Maintain threat model, asset inventory, dependency/SBOM inventory, privileged-account list, and security baseline.
- Centralize logs and time sync.
- Use offline patch staging and malware scanning.
- Run incident exercises.

Security and privacy controls

- MFA, segmentation, least privilege, key protection, immutable/offline backups, secure updates.
- No direct Internet on production servers.
- No shared privileged passwords.
- No silent security exceptions.

Acceptance / verification

- ✓ Credential revocation.
- ✓ Lateral movement denial.
- ✓ Restore after ransomware simulation.
- ✓ Vulnerability scan and remediation verification.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

18.3 Least privilege

Give each account only the API actions, database permissions, Fabric functions, host commands, and network paths required for its job. Separate CA, Fabric, database, OS, and backup admins.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Assume credentials and endpoints can be compromised.
- Layer preventive, detective, and recovery controls.
- Protect resources rather than trusting network location.
- Backups are part of security, not only operations.

Implementation blueprint

- Maintain threat model, asset inventory, dependency/SBOM inventory, privileged-account list, and security baseline.
- Centralize logs and time sync.
- Use offline patch staging and malware scanning.
- Run incident exercises.

Security and privacy controls

- MFA, segmentation, least privilege, key protection, immutable/offline backups, secure updates.
- No direct Internet on production servers.
- No shared privileged passwords.
- No silent security exceptions.

Acceptance / verification

- ✓ Credential revocation.
- ✓ Lateral movement denial.
- ✓ Restore after ransomware simulation.
- ✓ Vulnerability scan and remediation verification.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

18.4 Host hardening

Patch from vetted offline bundles, remove unused packages, disable password SSH where keys are used, restrict sudo, enforce host firewalls, collect logs, and monitor disk/CPU/service health.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Assume credentials and endpoints can be compromised.
- Layer preventive, detective, and recovery controls.
- Protect resources rather than trusting network location.
- Backups are part of security, not only operations.

Implementation blueprint

- Maintain threat model, asset inventory, dependency/SBOM inventory, privileged-account list, and security baseline.
- Centralize logs and time sync.
- Use offline patch staging and malware scanning.
- Run incident exercises.

Security and privacy controls

- MFA, segmentation, least privilege, key protection, immutable/offline backups, secure updates.
- No direct Internet on production servers.
- No shared privileged passwords.
- No silent security exceptions.

Acceptance / verification

- ✓ Credential revocation.
- ✓ Lateral movement denial.
- ✓ Restore after ransomware simulation.
- ✓ Vulnerability scan and remediation verification.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

18.5 Secrets management

Do not store passwords/private keys in source control or Compose files. Use root-owned secret files, OS keystores, HSM/smart cards where feasible, and strict backup/key-escrow procedures.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Assume credentials and endpoints can be compromised.
- Layer preventive, detective, and recovery controls.
- Protect resources rather than trusting network location.
- Backups are part of security, not only operations.

Implementation blueprint

- Maintain threat model, asset inventory, dependency/SBOM inventory, privileged-account list, and security baseline.
- Centralize logs and time sync.
- Use offline patch staging and malware scanning.
- Run incident exercises.

Security and privacy controls

- MFA, segmentation, least privilege, key protection, immutable/offline backups, secure updates.
- No direct Internet on production servers.
- No shared privileged passwords.
- No silent security exceptions.

Acceptance / verification

- ✓ Credential revocation.
- ✓ Lateral movement denial.
- ✓ Restore after ransomware simulation.
- ✓ Vulnerability scan and remediation verification.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

18.6 Ransomware resilience

Keep immutable/offline backup copies, restrict backup credentials, segment backup storage, test restoration, and document incident isolation. School systems are known ransomware targets, so offline architecture does not eliminate endpoint risk. [R14]

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Assume credentials and endpoints can be compromised.
- Layer preventive, detective, and recovery controls.
- Protect resources rather than trusting network location.
- Backups are part of security, not only operations.

Implementation blueprint

- Maintain threat model, asset inventory, dependency/SBOM inventory, privileged-account list, and security baseline.
- Centralize logs and time sync.
- Use offline patch staging and malware scanning.
- Run incident exercises.

Security and privacy controls

- MFA, segmentation, least privilege, key protection, immutable/offline backups, secure updates.
- No direct Internet on production servers.
- No shared privileged passwords.
- No silent security exceptions.

Acceptance / verification

- ✓ Credential revocation.
- ✓ Lateral movement denial.
- ✓ Restore after ransomware simulation.
- ✓ Vulnerability scan and remediation verification.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

18.7 Application security

Validate all inputs, use parameterized SQL, CSRF protection, secure cookies, anti-clickjacking headers, rate limits, centralized authorization, safe file handling, and dependency scanning on the staging machine.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Assume credentials and endpoints can be compromised.
- Layer preventive, detective, and recovery controls.
- Protect resources rather than trusting network location.
- Backups are part of security, not only operations.

Implementation blueprint

- Maintain threat model, asset inventory, dependency/SBOM inventory, privileged-account list, and security baseline.
- Centralize logs and time sync.
- Use offline patch staging and malware scanning.
- Run incident exercises.

Security and privacy controls

- MFA, segmentation, least privilege, key protection, immutable/offline backups, secure updates.
- No direct Internet on production servers.
- No shared privileged passwords.
- No silent security exceptions.

Acceptance / verification

- ✓ Credential revocation.
- ✓ Lateral movement denial.
- ✓ Restore after ransomware simulation.
- ✓ Vulnerability scan and remediation verification.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

18.8 Logging and alerting

Collect authentication failures, privilege changes, CA issuance/revocation, chaincode admin activity, database admin actions, backup failures, disk pressure, peer/orderer health, and repeated authorization denials.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Assume credentials and endpoints can be compromised.
- Layer preventive, detective, and recovery controls.
- Protect resources rather than trusting network location.
- Backups are part of security, not only operations.

Implementation blueprint

- Maintain threat model, asset inventory, dependency/SBOM inventory, privileged-account list, and security baseline.
- Centralize logs and time sync.
- Use offline patch staging and malware scanning.
- Run incident exercises.

Security and privacy controls

- MFA, segmentation, least privilege, key protection, immutable/offline backups, secure updates.
- No direct Internet on production servers.
- No shared privileged passwords.
- No silent security exceptions.

Acceptance / verification

- ✓ Credential revocation.
- ✓ Lateral movement denial.
- ✓ Restore after ransomware simulation.
- ✓ Vulnerability scan and remediation verification.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

18.9 Security acceptance gate

No production release without threat-model review, access-control tests, dependency inventory, backup/restore test, revoked-user test, certificate-expiry review, vulnerability scan, and signed risk acceptance for remaining issues.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Assume credentials and endpoints can be compromised.
- Layer preventive, detective, and recovery controls.
- Protect resources rather than trusting network location.
- Backups are part of security, not only operations.

Implementation blueprint

- Maintain threat model, asset inventory, dependency/SBOM inventory, privileged-account list, and security baseline.
- Centralize logs and time sync.
- Use offline patch staging and malware scanning.
- Run incident exercises.

Security and privacy controls

- MFA, segmentation, least privilege, key protection, immutable/offline backups, secure updates.
- No direct Internet on production servers.
- No shared privileged passwords.
- No silent security exceptions.

Acceptance / verification

- ✓ Credential revocation.
- ✓ Lateral movement denial.
- ✓ Restore after ransomware simulation.
- ✓ Vulnerability scan and remediation verification.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

19.1 Testing pyramid

Use chaincode unit tests, API tests, database tests, integration tests, full workflow tests, security tests, performance tests, and disaster-recovery tests. Production readiness is broader than contract unit-test coverage.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Test business controls and infrastructure failures, not only code paths.
- Keep test identities/data separate.
- Automate regression but retain human acceptance evidence.
- Recovery objectives must be measured.

Implementation blueprint

- Version test plans with release artifacts.
- Seed synthetic student data.
- Capture logs and transaction IDs for failures.
- Track defects to closure or accepted risk.

Security and privacy controls

- Never use production private keys in test.
- Sanitize copied production data.
- Protect test admin accounts.
- Treat performance tools as privileged.

Acceptance / verification

- ✓ Unit, integration, end-to-end, authorization, failure injection, load, backup/restore, and user acceptance.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

19.2 Chaincode unit tests

Test state transitions, authorization helpers, duplicate IDs, invalid versions, hash format, missing approvals, event payloads, and deterministic behavior.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Test business controls and infrastructure failures, not only code paths.
- Keep test identities/data separate.
- Automate regression but retain human acceptance evidence.
- Recovery objectives must be measured.

Implementation blueprint

- Version test plans with release artifacts.
- Seed synthetic student data.
- Capture logs and transaction IDs for failures.
- Track defects to closure or accepted risk.

Security and privacy controls

- Never use production private keys in test.
- Sanitize copied production data.
- Protect test admin accounts.
- Treat performance tools as privileged.

Acceptance / verification

- ✓ Unit, integration, end-to-end, authorization, failure injection, load, backup/restore, and user acceptance.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

19.3 Integration test network

Maintain a LAN test environment with the same organization/policy topology as production but separate certificates and data. Never use production private keys in test.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Test business controls and infrastructure failures, not only code paths.
- Keep test identities/data separate.
- Automate regression but retain human acceptance evidence.
- Recovery objectives must be measured.

Implementation blueprint

- Version test plans with release artifacts.
- Seed synthetic student data.
- Capture logs and transaction IDs for failures.
- Track defects to closure or accepted risk.

Security and privacy controls

- Never use production private keys in test.
- Sanitize copied production data.
- Protect test admin accounts.
- Treat performance tools as privileged.

Acceptance / verification

- ✓ Unit, integration, end-to-end, authorization, failure injection, load, backup/restore, and user acceptance.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

19.4 End-to-end scenarios

Automate enrollment -> attendance -> grade publish -> transcript -> credential issuance -> verification, including corrections and revocation.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Test business controls and infrastructure failures, not only code paths.
- Keep test identities/data separate.
- Automate regression but retain human acceptance evidence.
- Recovery objectives must be measured.

Implementation blueprint

- Version test plans with release artifacts.
- Seed synthetic student data.
- Capture logs and transaction IDs for failures.
- Track defects to closure or accepted risk.

Security and privacy controls

- Never use production private keys in test.
- Sanitize copied production data.
- Protect test admin accounts.
- Treat performance tools as privileged.

Acceptance / verification

✓ Unit, integration, end-to-end, authorization, failure injection, load, backup/restore, and user acceptance.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

19.5 Authorization tests

For each endpoint and chaincode function, test allow and deny cases for teacher, registrar, examiner, principal, auditor, student, and service identities.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Test business controls and infrastructure failures, not only code paths.
- Keep test identities/data separate.
- Automate regression but retain human acceptance evidence.
- Recovery objectives must be measured.

Implementation blueprint

- Version test plans with release artifacts.
- Seed synthetic student data.
- Capture logs and transaction IDs for failures.
- Track defects to closure or accepted risk.

Security and privacy controls

- Never use production private keys in test.
- Sanitize copied production data.
- Protect test admin accounts.
- Treat performance tools as privileged.

Acceptance / verification

- ✓ Unit, integration, end-to-end, authorization, failure injection, load, backup/restore, and user acceptance.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

19.6 Performance tests

Measure login, dashboard query, Fabric evaluate, transaction commit, batch attendance, grade-sheet publication, event indexing, report generation, and concurrent classroom usage.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Test business controls and infrastructure failures, not only code paths.
- Keep test identities/data separate.
- Automate regression but retain human acceptance evidence.
- Recovery objectives must be measured.

Implementation blueprint

- Version test plans with release artifacts.
- Seed synthetic student data.
- Capture logs and transaction IDs for failures.
- Track defects to closure or accepted risk.

Security and privacy controls

- Never use production private keys in test.
- Sanitize copied production data.
- Protect test admin accounts.
- Treat performance tools as privileged.

Acceptance / verification

- ✓ Unit, integration, end-to-end, authorization, failure injection, load, backup/restore, and user acceptance.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

19.7 Failure injection

Stop one orderer, one peer, PostgreSQL replica, DNS server, application node, and switch path in controlled tests. Verify defined degraded behavior and recovery.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Test business controls and infrastructure failures, not only code paths.
- Keep test identities/data separate.
- Automate regression but retain human acceptance evidence.
- Recovery objectives must be measured.

Implementation blueprint

- Version test plans with release artifacts.
- Seed synthetic student data.
- Capture logs and transaction IDs for failures.
- Track defects to closure or accepted risk.

Security and privacy controls

- Never use production private keys in test.
- Sanitize copied production data.
- Protect test admin accounts.
- Treat performance tools as privileged.

Acceptance / verification

✓ Unit, integration, end-to-end, authorization, failure injection, load, backup/restore, and user acceptance.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

19.8 Backup/restore drills

Restore Fabric ledger/MSP/CA, PostgreSQL, application config, file store, and identity provider into an isolated environment. Measure actual RPO/RTO instead of assuming backups work.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Test business controls and infrastructure failures, not only code paths.
- Keep test identities/data separate.
- Automate regression but retain human acceptance evidence.
- Recovery objectives must be measured.

Implementation blueprint

- Version test plans with release artifacts.
- Seed synthetic student data.
- Capture logs and transaction IDs for failures.
- Track defects to closure or accepted risk.

Security and privacy controls

- Never use production private keys in test.
- Sanitize copied production data.
- Protect test admin accounts.
- Treat performance tools as privileged.

Acceptance / verification

- ✓ Unit, integration, end-to-end, authorization, failure injection, load, backup/restore, and user acceptance.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

19.9 Acceptance evidence pack

For each release retain test plan, test results, screenshots/log extracts, package hashes, SBOM, config diff, security findings, recovery results, approvers, and known limitations.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Test business controls and infrastructure failures, not only code paths.
- Keep test identities/data separate.
- Automate regression but retain human acceptance evidence.
- Recovery objectives must be measured.

Implementation blueprint

- Version test plans with release artifacts.
- Seed synthetic student data.
- Capture logs and transaction IDs for failures.
- Track defects to closure or accepted risk.

Security and privacy controls

- Never use production private keys in test.
- Sanitize copied production data.
- Protect test admin accounts.
- Treat performance tools as privileged.

Acceptance / verification

- ✓ Unit, integration, end-to-end, authorization, failure injection, load, backup/restore, and user acceptance.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

20.1 Daily operations

Check node health, disk capacity, backup status, certificate expiry dashboard, failed transactions, event-indexer lag, login anomalies, and time synchronization.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Operations must work without Internet.
- Every backup must have a restore story.
- Monitor certificate expiry and storage before they become outages.
- Change production only through controlled releases.

Implementation blueprint

- Daily/weekly/monthly runbooks.
- Local monitoring and log retention.
- Versioned offline update bundles.
- Quarterly recovery drills and permission reviews.

Security and privacy controls

- Separate backup credentials.
- Offline copy rotation.
- Admin actions from management network.
- Key/certificate inventory and escrow.

Acceptance / verification

- ✓ Restore drill.
- ✓ Expired-cert simulation.
- ✓ Orderer/peer failure.
- ✓ Disk-full alarm.
- ✓ Rollback of failed upgrade.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

20.2 Monitoring stack

Run monitoring locally. Collect system metrics, Fabric peer/orderer metrics, database metrics, application health, logs, and alert rules without cloud telemetry.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Operations must work without Internet.
- Every backup must have a restore story.
- Monitor certificate expiry and storage before they become outages.
- Change production only through controlled releases.

Implementation blueprint

- Daily/weekly/monthly runbooks.
- Local monitoring and log retention.
- Versioned offline update bundles.
- Quarterly recovery drills and permission reviews.

Security and privacy controls

- Separate backup credentials.
- Offline copy rotation.
- Admin actions from management network.
- Key/certificate inventory and escrow.

Acceptance / verification

- ✓ Restore drill.
- ✓ Expired-cert simulation.
- ✓ Orderer/peer failure.
- ✓ Disk-full alarm.
- ✓ Rollback of failed upgrade.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

20.3 Backup architecture

Maintain online operational backups, a separate backup server/VLAN, and offline encrypted copies. Back up ledger data, MSP/CA material, PostgreSQL, Keycloak, config, file storage, and release artifacts.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Operations must work without Internet.
- Every backup must have a restore story.
- Monitor certificate expiry and storage before they become outages.
- Change production only through controlled releases.

Implementation blueprint

- Daily/weekly/monthly runbooks.
- Local monitoring and log retention.
- Versioned offline update bundles.
- Quarterly recovery drills and permission reviews.

Security and privacy controls

- Separate backup credentials.
- Offline copy rotation.
- Admin actions from management network.
- Key/certificate inventory and escrow.

Acceptance / verification

- ✓ Restore drill.
- ✓ Expired-cert simulation.
- ✓ Orderer/peer failure.
- ✓ Disk-full alarm.
- ✓ Rollback of failed upgrade.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

20.4 Fabric backup priorities

Protect ledger data and especially MSP/CA signing material. Fabric upgrade guidance explicitly begins with backing up ledger and MSPs. [R15]

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Operations must work without Internet.
- Every backup must have a restore story.
- Monitor certificate expiry and storage before they become outages.
- Change production only through controlled releases.

Implementation blueprint

- Daily/weekly/monthly runbooks.
- Local monitoring and log retention.
- Versioned offline update bundles.
- Quarterly recovery drills and permission reviews.

Security and privacy controls

- Separate backup credentials.
- Offline copy rotation.
- Admin actions from management network.
- Key/certificate inventory and escrow.

Acceptance / verification

- ✓ Restore drill.
- ✓ Expired-cert simulation.
- ✓ Orderer/peer failure.
- ✓ Disk-full alarm.
- ✓ Rollback of failed upgrade.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

20.5 PostgreSQL backup

Use tested logical and/or physical backups, transaction/WAL strategy as appropriate, separate credentials, retention tiers, and routine restore validation.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Operations must work without Internet.
- Every backup must have a restore story.
- Monitor certificate expiry and storage before they become outages.
- Change production only through controlled releases.

Implementation blueprint

- Daily/weekly/monthly runbooks.
- Local monitoring and log retention.
- Versioned offline update bundles.
- Quarterly recovery drills and permission reviews.

Security and privacy controls

- Separate backup credentials.
- Offline copy rotation.
- Admin actions from management network.
- Key/certificate inventory and escrow.

Acceptance / verification

- ✓ Restore drill.
- ✓ Expired-cert simulation.
- ✓ Orderer/peer failure.
- ✓ Disk-full alarm.
- ✓ Rollback of failed upgrade.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

20.6 Disaster-recovery runbook

Define total-site recovery order: network core, DNS/NTP, identity, CAs/keys, orderers, peers, database, file storage, API, indexer, frontend, monitoring, then validation/reconciliation.

Design rule. The ledger is only one control layer. Application authorization, database constraints, network segmentation, identity lifecycle, backups, and human approval remain equally important.

Design intent

- Operations must work without Internet.
- Every backup must have a restore story.
- Monitor certificate expiry and storage before they become outages.
- Change production only through controlled releases.

Implementation blueprint

- Daily/weekly/monthly runbooks.
- Local monitoring and log retention.
- Versioned offline update bundles.
- Quarterly recovery drills and permission reviews.

Security and privacy controls

- Separate backup credentials.
- Offline copy rotation.
- Admin actions from management network.
- Key/certificate inventory and escrow.

Acceptance / verification

- ✓ Restore drill.
- ✓ Expired-cert simulation.
- ✓ Orderer/peer failure.
- ✓ Disk-full alarm.
- ✓ Rollback of failed upgrade.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

20.7 Certificate expiry and rotation

Inventory every TLS, enrollment, signing, and web certificate with owner/expiry. Rotate early in a maintenance window and test both old/new trust chains when applicable.

Design rule. For every design choice, record what data is authoritative, what evidence is immutable, who can correct errors, and how the school proves that a correction was legitimate.

Design intent

- Operations must work without Internet.
- Every backup must have a restore story.
- Monitor certificate expiry and storage before they become outages.
- Change production only through controlled releases.

Implementation blueprint

- Daily/weekly/monthly runbooks.
- Local monitoring and log retention.
- Versioned offline update bundles.
- Quarterly recovery drills and permission reviews.

Security and privacy controls

- Separate backup credentials.
- Offline copy rotation.
- Admin actions from management network.
- Key/certificate inventory and escrow.

Acceptance / verification

- ✓ Restore drill.
- ✓ Expired-cert simulation.
- ✓ Orderer/peer failure.
- ✓ Disk-full alarm.
- ✓ Rollback of failed upgrade.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

20.8 Offline upgrade process

On the staging machine, download releases, advisories, container images, dependencies, and checksums. Test in an isolated clone, approve, import into local registry/repository, then perform a controlled production rollout.

Design rule. Prefer deterministic identifiers and explicit version fields. Operational convenience must never depend on hidden state, manual database edits, or an administrator remembering undocumented exceptions.

Design intent

- Operations must work without Internet.
- Every backup must have a restore story.
- Monitor certificate expiry and storage before they become outages.
- Change production only through controlled releases.

Implementation blueprint

- Daily/weekly/monthly runbooks.
- Local monitoring and log retention.
- Versioned offline update bundles.
- Quarterly recovery drills and permission reviews.

Security and privacy controls

- Separate backup credentials.
- Offline copy rotation.
- Admin actions from management network.
- Key/certificate inventory and escrow.

Acceptance / verification

- ✓ Restore drill.
- ✓ Expired-cert simulation.
- ✓ Orderer/peer failure.
- ✓ Disk-full alarm.
- ✓ Rollback of failed upgrade.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Implementation Detail

20.9 Operational maturity

Run quarterly restore drills, annual architecture review, term-based permission review, monthly privileged-activity review, periodic staff training, and documented decommissioning of retired nodes/media.

Design rule. Implementation should be documented as a reproducible local procedure: prerequisites, configuration files, owner, input data, expected output, failure modes, and rollback steps.

Design intent

- Operations must work without Internet.
- Every backup must have a restore story.
- Monitor certificate expiry and storage before they become outages.
- Change production only through controlled releases.

Implementation blueprint

- Daily/weekly/monthly runbooks.
- Local monitoring and log retention.
- Versioned offline update bundles.
- Quarterly recovery drills and permission reviews.

Security and privacy controls

- Separate backup credentials.
- Offline copy rotation.
- Admin actions from management network.
- Key/certificate inventory and escrow.

Acceptance / verification

- ✓ Restore drill.
- ✓ Expired-cert simulation.
- ✓ Orderer/peer failure.
- ✓ Disk-full alarm.
- ✓ Rollback of failed upgrade.

Deliverable for this topic: Produce a one-page configuration or workflow record containing owner, inputs, roles, data classification, ledger evidence, off-chain system of record, failure behavior, and test evidence. Store it with the project's controlled documentation.

Code / Configuration

21.1 Repository Structure

Use a monorepo or clearly versioned multi-repo layout so chaincode, API, frontend, infrastructure, docs, and tests are released together.

```
school-blockchain/
├── infrastructure/
│   ├── fabric/           # orderers, peers, CAs, channel config
│   ├── identity/        # local IdP configuration exports
│   ├── database/        # PostgreSQL migrations and backup jobs
│   └── monitoring/      # local dashboards and alert rules
├── chaincode/
│   └── school-ledger-ts/
├── services/
│   ├── api/
│   └── indexer/
├── web/
├── tests/
│   ├── integration/
│   ├── e2e/
│   └── recovery/
├── docs/
│   ├── architecture/
│   ├── runbooks/
│   └── decisions/
├── release/
└── offline-bundles/
```

Implementation notes

- Keep generated certificates and secrets out of source control.
- Version database migrations and JSON schemas.
- Each release directory should include hashes/SBOM/test evidence.
- Use a separate secure backup for CA/MSP material.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

Code / Configuration

21.2 Example Offline Environment Variables

Centralize non-secret runtime settings and inject secrets from protected files or keystores.

```
APP_ENV=production
APP_BASE_URL=https://app.school.lan
IDP_ISSUER=https://id.school.lan/realms/school
DB_HOST=db.school.lan
DB_PORT=5432
DB_NAME=school
FABRIC_CHANNEL=schoolchannel
FABRIC_CHAINCODE=schoolledger
FABRIC_GATEWAY=peer0.registrar.school.lan:7051
FABRIC_MSP_ID=RegistrarMSP
FABRIC_TLS_CA=/run/secrets/registrar_peer_tls_ca.pem
FABRIC_CERT=/run/secrets/api_client_cert.pem
FABRIC_KEY=/run/secrets/api_client_key.pem
NTP_SERVERS=10.10.10.20,10.10.10.21
LOG_LEVEL=info
```

Implementation notes

- Never put database passwords or private keys in .env committed to Git.
- Use school.lan names resolved by local DNS.
- No external telemetry endpoint should appear in production settings.
- Validate all required variables at service startup.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

Code / Configuration

21.3 Example Three-Orderer Compose Skeleton

Deploy each orderer on a different physical host; this combined example is for understanding the configuration shape.

```
services:
  orderer1:
    image: registry.school.lan/hyperledger/fabric-orderer@sha256:<PINNED>
    environment:
      - FABRIC_CFG_PATH=/etc/hyperledger/fabric
      - ORDERER_GENERAL_LISTENADDRESS=0.0.0.0
      - ORDERER_GENERAL_LISTENPORT=7050
      - ORDERER_GENERAL_LOCALMSPID=OrdererMSP
      - ORDERER_GENERAL_TLS_ENABLED=true
    volumes:
      - ./orderer1/msp:/var/hyperledger/orderer/msp:ro
      - ./orderer1/tls:/var/hyperledger/orderer/tls:ro
      - orderer1_data:/var/hyperledger/production/orderer
    networks: [blockchain]
# orderer2 and orderer3: same pattern, unique certs/data/hosts
networks:
  blockchain: { internal: true }
volumes:
  orderer1_data:
```

Implementation notes

- Use internal networks and firewall rules rather than published ports to every VLAN.
- Set resource limits and log rotation.
- Back up/orderer data according to the runbook.
- Test one-orderer outage while maintaining quorum.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

Code / Configuration

21.4 Example Peer Compose Skeleton

Peers host ledger/world-state, Gateway, endorsement, and chaincode execution for each organization.

```
services:
  peer0-registrar:
    image: registry.school.lan/hyperledger/fabric-peer@sha256:<PINNED>
    environment:
      - CORE_PEER_ID=peer0.registrar.school.lan
      - CORE_PEER_ADDRESS=peer0.registrar.school.lan:7051
      - CORE_PEER_LOCALMSPID=RegistrarMSP
      - CORE_PEER_TLS_ENABLED=true
      - CORE_PEER_GOSSIP_EXTERNALENDPOINT=peer0.registrar.school.lan:7051
    volumes:
      - ./msp:/etc/hyperledger/fabric/msp:ro
      - ./tls:/etc/hyperledger/fabric/tls:ro
      - peer_registrar_data:/var/hyperledger/production
    networks: [blockchain]
volumes:
  peer_registrar_data:
```

Implementation notes

- Give each organization its own MSP and peer data volume.
- Expose Gateway only to approved application hosts.
- Monitor ledger height and resource use.
- Keep organization admin credentials outside the peer container.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

Code / Configuration

21.5 Example Organization and Policy Concepts

The exact configtx.yaml is site-specific; this skeleton shows the governance intent.

```
Organizations:
- &Registrar
  Name: RegistrarMSP
  ID: RegistrarMSP
  MSPDir: crypto/registrar/msp
  Policies:
    Readers: Type: Signature, Rule: "OR('RegistrarMSP.admin','RegistrarMSP.peer','RegistrarMSP.client')"
    Writers: Type: Signature, Rule: "OR('RegistrarMSP.admin','RegistrarMSP.client')"
    Admins:  Type: Signature, Rule: "OR('RegistrarMSP.admin')"
    Endorsement: Type: Signature, Rule: "OR('RegistrarMSP.peer')"
# Repeat for AcademicMSP and AdministrationMSP.
```

Implementation notes

- Use Node OU roles and validated MSP certificates.
- Keep lifecycle/config admin rights narrow.
- Review default majority policies against real school governance.
- Record every production config update and signatures.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

Code / Configuration

21.6 Transaction Endorsement Matrix

Write business policy first; implement chaincode/channel policy second.

Transaction	Required organizations
StudentEnrollmentApprove	Registrar + Administration
AttendanceFinalize	Academic
AttendanceLateCorrection	Academic + Administration (after cutoff)
GradeSheetPublish	Academic + Registrar
GradeSheetCorrect	Academic + Registrar
CredentialIssue	Registrar + Administration
CredentialRevoke	Registrar + Administration
PolicyChange	Administration + affected organization

Implementation notes

- Use stronger endorsement only where the business decision requires it.
- Avoid all-three endorsements for every low-risk event; it reduces availability.
- Document emergency exceptions separately.
- Test each allow/deny case with real organization identities.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

Code / Configuration

21.7 Private Data Collection Skeleton

Use PDCs narrowly when sensitive data truly needs Fabric distribution to selected organizations.

```
[
  {
    "name": "registrarPrivate",
    "policy": "OR('RegistrarMSP.member', 'AdministrationMSP.member')",
    "requiredPeerCount": 1,
    "maxPeerCount": 2,
    "blockToLive": 0,
    "memberOnlyRead": true,
    "memberOnlyWrite": true
  }
]
```

Implementation notes

- Default student PII should remain in PostgreSQL, not a PDC.
- A PDC still needs retention, access, backup, and purge decisions.
- Hash public evidence only when it supports a clear audit use case.
- Collection policy changes require careful upgrade/governance review.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

Code / Configuration

21.8 Chaincode Type Definitions

Use small deterministic ledger records; detailed student data lives off-chain.

```
export interface LedgerEnvelope {
  schemaVersion: number;
  entityType: 'STUDENT'|'ATTENDANCE'|'GRADE_SHEET'|'CREDENTIAL';
  entityId: string;
  version: number;
  contentHash: string;           // sha256 hex of canonical off-chain record
  status: string;
  actorId: string;              // pseudonymous staff identifier
  organization: string;
  reasonCode?: string;
  correlationId: string;
  createdAt: string;           // server-supplied ISO timestamp policy
}
```

Implementation notes

- Validate enum values and hash format.
- Do not accept organization/role directly from browser input.
- Keep schemaVersion explicit.
- Use deterministic serialization for any value used by chaincode logic.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

Code / Configuration

21.9 Student Chaincode Skeleton

The ledger stores approved enrollment evidence, not the full student profile.

```
@Transaction()
public async approveEnrollment(
  ctx: Context,
  studentUid: string,
  profileHash: string,
  correlationId: string
): Promise<void> {
  this.requireRole(ctx, 'registrar');
  await this.rejectDuplicateRequest(ctx, correlationId);
  const key = ctx.stub.createCompositeKey('student', [studentUid]);
  const existing = await ctx.stub.getState(key);
  if (existing.length) throw new Error('STUDENT_EXISTS');
  const record = {
    schemaVersion: 1, studentUid, version: 1,
    profileHash, status: 'ACTIVE', correlationId
  };
  await ctx.stub.putState(key, Buffer.from(JSON.stringify(record)));
  ctx.stub.setEvent('StudentEnrollmentApproved', Buffer.from(JSON.stringify({studentUid, correlationId})));
}
```

Implementation notes

- Implement requireRole from verified client identity attributes/MSP.
- Validate hash and ID format before write.
- Use idempotency storage for correlationId.
- Keep actual name/DOB/contact data out of this state.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

Code / Configuration

21.10 Attendance Contract Skeleton

Attendance can use compact finalized-session evidence while detailed rows remain in PostgreSQL.

```
@Transaction()
public async finalizeAttendance(
  ctx: Context,
  sessionId: string,
  rosterHash: string,
  revision: number,
  correlationId: string
) {
  this.requireRole(ctx, 'teacher');
  const key = ctx.stub.createCompositeKey('attendance', [sessionId, String(revision)]);
  if ((await ctx.stub.getState(key)).length) throw new Error('REVISION_EXISTS');
  const record = { sessionId, rosterHash, revision, status:'FINAL', correlationId };
  await ctx.stub.putState(key, Buffer.from(JSON.stringify(record)));
  ctx.stub.setEvent('AttendanceFinalized', Buffer.from(JSON.stringify(record)));
}
```

Implementation notes

- The API verifies that the teacher is assigned to the session.
- After correction cutoff, require a stronger approval path.
- Database rows and ledger rosterHash must reconcile.
- Use a canonical sorted roster before hashing.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

Code / Configuration

21.11 Grade Publication Contract Skeleton

Publish only the approved grade-sheet hash and evidence needed for governance.

```
@Transaction()
public async publishGradeSheet(
  ctx: Context,
  courseId: string,
  termId: string,
  sheetHash: string,
  version: number,
  correlationId: string
) {
  this.requireApprovedPublisher(ctx);
  const key = ctx.stub.createCompositeKey('gradeSheet', [courseId, termId, String(version)]);
  if ((await ctx.stub.getState(key)).length) throw new Error('VERSION_EXISTS');
  const rec = {courseId, termId, sheetHash, version, status: 'PUBLISHED', correlationId};
  await ctx.stub.putState(key, Buffer.from(JSON.stringify(rec)));
  ctx.stub.setEvent('GradeSheetPublished', Buffer.from(JSON.stringify(rec)));
}
```

Implementation notes

- Configure endorsement to reflect Academic + Registrar approval.
- Exact student grades remain off-chain.
- Corrections reference prior version and reason.
- Transcript generation uses only approved versions.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

Code / Configuration

21.12 Credential Contract Skeleton

Credentials need issuance, lookup, supersession, and revocation states.

```
@Transaction()
public async issueCredential(ctx: Context, id:string, holderUid:string,
  credentialHash:string, keyId:string, correlationId:string) {
  this.requireCredentialIssuer(ctx);
  const key = ctx.stub.createCompositeKey('credential',[id]);
  if ((await ctx.stub.getState(key)).length) throw new Error('CREDENTIAL_EXISTS');
  const rec = {id,holderUid,credentialHash,keyId,status:'VALID',correlationId};
  await ctx.stub.putState(key, Buffer.from(JSON.stringify(rec)));
  ctx.stub.setEvent('CredentialIssued', Buffer.from(JSON.stringify({id,status:'VALID'})));
}

@Transaction()
public async revokeCredential(ctx: Context, id:string, reasonCode:string) {
  this.requireCredentialIssuer(ctx);
  // load, change status to REVOKED, keep history through ledger transaction
}
```

Implementation notes

- Never delete issuance history to 'revoke' a credential.
- Separate document-signing key from Fabric CA key.
- Verification API must be read-only.
- Define status semantics: VALID, SUPERSEDED, REVOKED.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

Code / Configuration

21.13 Access-Control Helper Concept

Chaincode should derive authorization from verified identity context, not request JSON.

```
private requireMsp(ctx: Context, allowed: string[]) {
  const msp = ctx.clientIdentity.getMSPID();
  if (!allowed.includes(msp)) throw new Error('FORBIDDEN_MSP');
}

private requireAttr(ctx: Context, name: string, value: string) {
  if (!ctx.clientIdentity.assertAttributeValue(name, value)) {
    throw new Error(`FORBIDDEN_${name.toUpperCase()}`);
  }
}

private requireCredentialIssuer(ctx: Context) {
  this.requireMsp(ctx, ['RegistrarMSP', 'AdministrationMSP']);
  this.requireAttr(ctx, 'schoolRole', 'credential-issuer');
}
```

Implementation notes

- Use MSP + attributes only after formal certificate issuance policy.
- Do not encode fragile org charts directly into every function.
- Centralize authorization helpers and test them thoroughly.
- Application authorization remains necessary in addition to chaincode checks.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

Code / Configuration

21.14 Chaincode Test Examples

Test deterministic business rules with synthetic data and identities.

```
describe('Student enrollment', () => {
  it('rejects duplicate student', async () => {
    await contract.approveEnrollment(ctx, 'stu-001', hashA, 'req-1');
    await expect(contract.approveEnrollment(ctx, 'stu-001', hashB, 'req-2'))
      .rejects.toThrow('STUDENT_EXISTS');
  });

  it('rejects unauthorized MSP', async () => {
    mockIdentity.mspId = 'StudentMSP';
    await expect(contract.approveEnrollment(ctx, 'stu-002', hashA, 'req-3'))
      .rejects.toThrow();
  });
});
```

Implementation notes

- Mock client identity and transient data carefully.
- Test event payloads and state bytes.
- Add integration tests against a real multi-org test network.
- Test chaincode upgrade against existing state fixtures.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

Code / Configuration

21.15 Node.js Fabric Gateway Connection

The backend uses a protected application identity to connect to a peer Gateway.

```
import * as grpc from '@grpc/grpc-js';
import { connect, hash, signers } from '@hyperledger/fabric-gateway';

const client = new grpc.Client(
  process.env.FABRIC_GATEWAY!,
  grpc.credentials.createSsl(readTlsCa())
);
const gateway = connect({
  client,
  identity: { mspId: 'RegistrarMSP', credentials: readCert() },
  signer: signers.newPrivateKeySigner(readPrivateKey()),
  hash: hash.sha256,
});
const network = gateway.getNetwork('schoolchannel');
const contract = network.getContract('schoolledger');
```

Implementation notes

- Prefer per-user transaction identities for high-assurance workflows where feasible.
- Keep private keys out of source and environment dumps.
- Use timeouts and retry only with idempotency.
- Close gRPC/gateway resources cleanly.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

Code / Configuration

21.16 REST Mutation Endpoint Pattern

Validate business data, persist draft/approved data, then submit the anchor transaction with a correlation ID.

```
app.post('/api/v1/enrollments/:uid/approve', requireRole('registrar'), async (req,res) => {
  const requestId = requireUuid(req.headers['idempotency-key']);
  const student = await repo.loadForApproval(req.params.uid);
  validateStudent(student);
  const canonical = canonicalize(student.auditProjection);
  const profileHash = sha256(canonical);
  await transactionService.submitAndWait('approveEnrollment', [
    student.uid, profileHash, requestId
  ]);
  await repo.markAnchored(student.uid, profileHash, requestId);
  res.json({status:'committed', requestId, profileHash});
});
```

Implementation notes

- Use a database transaction/outbox pattern to manage partial failures.
- Never trust role values sent by the browser.
- Return a clear committed/failed result.
- Log request ID and Fabric transaction ID together.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

Code / Configuration

21.17 PostgreSQL Core Schema Skeleton

Keep PII, workflow state, and reporting-friendly rows in a relational schema.

```
CREATE TABLE student (  
    student_uid uuid PRIMARY KEY,  
    legal_name text NOT NULL,  
    date_of_birth date NOT NULL,  
    status text NOT NULL,  
    profile_version integer NOT NULL DEFAULT 1,  
    profile_hash char(64),  
    ledger_tx_id text,  
    created_at timestamptz NOT NULL DEFAULT now(),  
    updated_at timestamptz NOT NULL DEFAULT now()  
);  
  
CREATE TABLE ledger_anchor (  
    request_id uuid PRIMARY KEY,  
    entity_type text NOT NULL,  
    entity_id text NOT NULL,  
    version integer NOT NULL,  
    content_hash char(64) NOT NULL,  
    fabric_tx_id text UNIQUE,  
    block_number bigint,  
    commit_status text NOT NULL,  
    created_at timestamptz NOT NULL DEFAULT now()  
);
```

Implementation notes

- Encrypt storage and backups.
- Use migrations, not manual schema edits.
- Create indexes for operational queries, not in blockchain state.
- Restrict direct database write permissions to services/migration roles.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

Code / Configuration

21.18 Event Indexer Checkpoint Pattern

Index committed events into PostgreSQL and resume safely after service restart.

```
async function processBlock(blockNumber: bigint) {
  const events = await fabricEvents.readBlock(blockNumber);
  await db.transaction(async tx => {
    for (const event of events) {
      await tx.query(`INSERT INTO fabric_event(tx_id,event_name,payload,block_no)
        VALUES ($1,$2,$3,$4) ON CONFLICT (tx_id,event_name) DO NOTHING`,
        [event.txId,event.name,event.payload,blockNumber]);
    }
    await tx.query(`UPDATE indexer_checkpoint SET last_block=$1 WHERE id=1`,[blockNumber]);
  });
}
```

Implementation notes

- Use unique constraints for idempotent replay.
- Advance checkpoint in the same DB transaction as indexed events.
- Alert when indexer lags behind peer ledger height.
- Rebuild read models from ledger/events when necessary.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

Code / Configuration

21.19 Local Web Security Headers and Proxy

Terminate TLS locally and apply safe defaults at the reverse proxy.

```
server {
    listen 443 ssl http2;
    server_name app.school.lan;
    ssl_certificate      /etc/school/tls/app.crt;
    ssl_certificate_key  /etc/school/tls/app.key;

    add_header Content-Security-Policy "default-src 'self'; frame-ancestors 'none'" always;
    add_header X-Content-Type-Options nosniff always;
    add_header Referrer-Policy no-referrer always;

    location /api/ {
        proxy_pass http://school-api:8080;
        proxy_set_header X-Request-ID $request_id;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

Implementation notes

- Use locally trusted web PKI.
- Keep static assets local under 'self'.
- HSTS is useful when the local domain and certificate lifecycle are stable.
- Restrict management endpoints to the management VLAN.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

Code / Configuration

21.20 Backup Manifest and Restore Verification

A backup is complete only when all critical layers and cryptographic dependencies are included and restore-tested.

```
backup-set/YYYY-MM-DD/
├── manifest.json           # versions, hashes, hostnames, timestamps
├── fabric/
│   ├── peer-ledger/
│   ├── orderer-ledger/
│   └── channel-config/
├── crypto/
│   ├── msp/
│   ├── ca-db-config/
│   └── encrypted-key-escrow/
├── postgres/
│   └── school.dump
├── identity/
│   └── realm-export-and-db/
├── files/
│   └── documents-snapshot/
├── app/
│   └── configuration-and-release-bundle/
└── verification/
    └── restore-test-results.txt
```

Implementation notes

- Store at least one encrypted copy offline from normal admin credentials.
- Record hash for every backup component.
- Perform restores to an isolated network on a schedule.
- Document RPO/RTO actually achieved during drills.

Production rule: customize names, paths, credentials, policies, ports, resource limits, certificate paths, and backup locations. Do not paste examples into production without review and testing.

22.1 Phased Implementation Roadmap

A safe school deployment should progress through controlled gates rather than a big-bang launch.

Phase	Duration example	Deliverables	Gate
0 - Discovery	2-4 weeks	Scope, stakeholders, data map, governance, acceptance criteria	Principal/IT/privacy approval
1 - Lab	3-6 weeks	Local test network, sample chaincode, synthetic data	Architecture review
2 - Pilot	4-8 weeks	One grade/year or department, backup/monitoring, training	Pilot acceptance
3 - Production core	4-8 weeks	Enrollment, attendance evidence, grades, credentials	Go-live board
4 - Expansion	Per term	Optional modules/integrations	Module privacy/security review

Rule: do not add another module until backup/restore, identity revocation, and current-module reconciliation are demonstrably working.

22.2 Pilot Acceptance Checklist

- ✓ Pilot users are real staff but test/student data is synthetic or explicitly authorized.
- ✓ One orderer can fail without stopping commits; peers recover after restart.
- ✓ Unauthorized teacher/registrar/principal actions are denied and logged.
- ✓ Enrollment, attendance, grade publication, correction, certificate issue and revocation complete end-to-end.
- ✓ Database-to-ledger reconciliation reports zero unexplained mismatches.
- ✓ All production-bound software artifacts are available in the offline bundle/local registry.
- ✓ Backup restore succeeds into an isolated environment.
- ✓ Certificate expiry dashboard and alerting are working.
- ✓ No production page makes an Internet request when inspected with browser/network tools.
- ✓ Staff can perform common workflows without understanding Fabric internals.
- ✓ Privacy review confirms no prohibited PII appears in ledger keys, values, or events.
- ✓ Pilot sign-off records known limitations and a rollback plan.

22.3 Daily, Weekly, Monthly Operations SOP

Daily

- Service health
- Disk usage
- Backup job result
- Indexer lag
- Failed auth spikes
- Time sync

Weekly

- Restore sample
- Certificate expiry review
- Privileged transaction review
- Database/ledger reconciliation
- Capacity trend

Monthly / Term

- Permission review
- Offline backup rotation
- Patch/advisory staging
- Change board
- Recovery exercise
- Training refresh

Shift handover record

Record open incidents, disabled nodes, expiring certificates, pending change tickets, failed backups, storage warnings, unresolved reconciliation mismatches, and temporary policy exceptions.

Never normalize alerts. A recurring failed backup or certificate warning is a production defect, not background noise.

22.4 Incident Response for a Local School Blockchain

1. **Identify:** confirm event source, affected users/services, timestamps, and whether student data or cryptographic keys may be involved.
2. **Contain:** isolate compromised endpoint/VLAN/account. Do not destroy logs or immediately wipe systems.
3. **Preserve evidence:** export logs, transaction IDs, host state, certificate serials, database audit data, and firewall events.
4. **Revoke:** disable identity sessions and revoke relevant certificates/keys where compromise is plausible.
5. **Recover:** restore affected services from known-good artifacts/backups; validate ledger/database consistency.
6. **Communicate:** follow school/legal privacy notification rules and leadership escalation.
7. **Learn:** update threat model, controls, tests, and training.

Ransomware-specific priority

Protect backup credentials and offline copies; rebuild from clean release artifacts rather than trusting a compromised host. CISA publishes K-12 ransomware resources that can be mapped into the school's local incident plan. [R14]

22.5 Certificate and Key Rotation SOP

Key/certificate	Owner	Rotation trigger	Special rule
Fabric root CA	Security governance	Long lifecycle / compromise	Keep offline; replacement is planned ceremony
Intermediate/enrollment CA	CA admin	Policy/expiry/compromise	Back up DB/config/key separately
Peer/orderer TLS	Fabric admin	Before expiry	Coordinate trust chain and restart
Staff client cert	Identity admin	Expiry/role change/loss	Revoke immediately on departure
Web TLS	Infrastructure admin	Before expiry	Update local trust deployment if CA changes
Credential signing key	Registrar/security	Scheduled/compromise	Keep historical public keys for old credential verification

Inventory fields: key ID, purpose, owner, storage location, public certificate, serial/fingerprint, issued date, expiry, rotation date, revocation status, backup/escrow location.

22.6 Staff Training Plan

Audience	Training	Practical assessment
Teachers	Login/MFA, attendance, grade drafts, corrections, phishing/device hygiene	Complete class attendance and correction
Registrar	Enrollment, transcript/certificate workflows, privacy, verification	Issue and revoke a test credential
Principal/exam office	Approvals, exceptions, audit evidence	Approve/reject test publication
IT operations	Services, logs, backup, restore, certificates, offline updates	Recover a failed node/service
Security/CA admins	PKI, revocation, incident handling, privileged access	Rotate/revoke a test identity
Auditors	Transaction/evidence reading, reconciliation, access logs	Reconstruct a sample correction history

Repeat short refreshers each term and after major workflow changes. Training should teach the school process first; Fabric terminology is only required for technical staff.

22.7 Example Bill of Materials and Cost Drivers

Component	Small-school baseline	Primary cost driver
Compute hosts	3 physical servers + optional spare	RAM, SSD/NVMe, warranty, redundancy
Network	Managed switches + firewall/router	VLAN capacity, redundant uplinks
Power	UPS for servers/network + spare batteries	Runtime and battery replacement
Backup	Separate NAS/server + encrypted offline media	Retention and restore performance
Admin security	Dedicated management workstation + hardware MFA keys	Number of privileged users
Software	Mostly open-source stack	Engineering, maintenance, support, training

Budget warning: the dominant cost is usually not blockchain software licensing. It is reliable infrastructure, secure operations, staff time, integration, testing, training, backups, and long-term maintenance.

22.8 Core Glossary A-M

Anchor - A ledger record that stores a hash/reference proving the state of an off-chain record.

Block - An ordered batch of transactions in the ledger.

CA - Certificate Authority that issues and revokes cryptographic identities.

Canonicalization - Rules that make the same logical record serialize identically before hashing.

Chaincode - Fabric smart-contract package containing business logic.

Channel - A Fabric communication/ledger scope with defined member organizations.

Commit - The step where an ordered transaction is validated and written to the ledger.

Consensus - Process used by ordering nodes to agree on transaction order.

Correlation ID - Identifier linking UI/API/database/ledger logs for one business request.

CRL - Certificate Revocation List.

Endorsement - Peer execution/signature showing a transaction proposal satisfies policy.

Fabric Gateway - Peer service/client API path for simplified application transaction submission.

Hash - Fixed-size digest used to detect content changes.

HSM - Hardware Security Module for protected key operations.

Idempotency - Property that safe retries do not create duplicate logical actions.

Ledger - Fabric blockchain history plus current world-state view.

MSP - Membership Service Provider defining trusted identities and roles.

Orderer - Fabric node that participates in ordering transactions into blocks.

PDC - Private Data Collection shared only with authorized Fabric organizations.

Peer - Fabric node that hosts ledger, world state, Gateway and chaincode endorsement.

22.9 Core Glossary N-Z

PII - Personally identifiable information; minimize immutable placement.

PKI - Public Key Infrastructure for certificates, keys, trust and revocation.

Raft - Crash-fault-tolerant leader/follower ordering protocol commonly used by Fabric.

RBAC - Role-Based Access Control.

RPO - Recovery Point Objective - acceptable data loss window.

RTO - Recovery Time Objective - acceptable recovery duration.

SBOM - Software Bill of Materials listing components/dependencies.

State-based endorsement - Endorsement rule attached to specific ledger keys/state.

StudentUID - Pseudonymous stable internal identifier used in ledger references.

TLS - Transport Layer Security for encrypted/authenticated network connections.

Transaction ID - Unique identifier for a Fabric transaction.

Version - Monotonic revision number used to preserve correction history.

VLAN - Virtual LAN used to isolate network zones.

World state - Current key/value state derived from committed ledger transactions.

X.509 - Certificate standard used by Fabric's default MSP identity model.

Zero trust - Security approach where network location alone does not grant implicit trust.

Working vocabulary: The implementation team should agree on one meaning for terms like "final", "published", "approved", "revoked", "archived", and "deleted". Ambiguous business words create inconsistent code and audit evidence.

22.10 Primary References and Final Go-Live Gate

Primary references

[R1] Hyperledger Fabric Docs - Introduction / permissioned model
<https://hyperledger-fabric.readthedocs.io/en/latest/whatis.html>

[R2] Hyperledger Fabric v3.1 / release notes; v3.1.5 current as checked 2026-08
<https://github.com/hyperledger/fabric/releases>

[R3] Hyperledger Fabric - Ordering Service (Raft and ordering concepts)
https://hyperledger-fabric.readthedocs.io/en/latest/orderer/ordering_service.html

[R4] Hyperledger Fabric - Fabric Gateway
<https://hyperledger-fabric.readthedocs.io/en/latest/gateway.html>

[R5] Hyperledger Fabric - Membership Service Provider
<https://hyperledger-fabric.readthedocs.io/en/latest/membership/membership.html>

[R6] Hyperledger Fabric - Security Model
https://hyperledger-fabric.readthedocs.io/en/latest/security_model.html

[R7] Hyperledger Fabric - Private Data
<https://hyperledger-fabric.readthedocs.io/en/latest/private-data-arch.html>

[R8] Hyperledger Fabric - Chaincode Lifecycle
https://hyperledger-fabric.readthedocs.io/en/latest/chaincode_lifecycle.html

[R9] Hyperledger Fabric - Endorsement Policies
<https://hyperledger-fabric.readthedocs.io/en/latest/endorsement-policies.html>

[R10] Hyperledger Fabric - Ledger
<https://hyperledger-fabric.readthedocs.io/en/latest/ledger/ledger.html>

[R11] Hyperledger Fabric - Installation
<https://hyperledger-fabric.readthedocs.io/en/latest/install.html>

[R12] U.S. Department of Education - FERPA / Protecting Student Privacy
<https://studentprivacy.ed.gov/ferpa>

[R13] NIST SP 800-207 - Zero Trust Architecture
<https://csrc.nist.gov/pubs/sp/800/207/final>

[R14] CISA - K-12 resources / ransomware guidance
<https://www.cisa.gov/stopransomware/k-12-resources>

[R15] Hyperledger Fabric - Upgrading / backup ledger and MSP guidance
<https://hyperledger-fabric.readthedocs.io/en/latest/upgrade.html>

[R16] NIST Cybersecurity Framework 2.0
<https://www.nist.gov/cyberframework>

[R17] PostgreSQL 18 current documentation
<https://www.postgresql.org/docs/current/>

[R18] Keycloak documentation - local server installation and administration
<https://www.keycloak.org/documentation>

Final go-live gate

- ✓ Scope and governance formally approved.
- ✓ No prohibited PII found on-chain.
- ✓ Three-orderer quorum/failover tested.
- ✓ Role and endorsement matrix tested.
- ✓ Revoked staff access denied.
- ✓ Database/ledger reconciliation clean.
- ✓ Credential issue/revoke/verify tested.
- ✓ Backups restored successfully.
- ✓ All production dependencies available offline.
- ✓ No Internet calls from production application.
- ✓ Monitoring and certificate-expiry alerts active.
- ✓ Incident and disaster-recovery runbooks exercised.
- ✓ Staff training completed.
- ✓ Known risks signed off by accountable leadership.
- ✓ Rollback plan and maintenance window documented.

Implementation endpoint: The system is ready when the school can operate, audit, recover, rotate identities, and update it without relying on the Internet or on undocumented knowledge held by one developer.